MODY UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Engineering and Technology

DAA LAB

LAB RECORD

Student Name: Priya

Enrollment Number:240157

Faculty Name: Dr. P. K. Bishnoi

Mody University of Science and Technology

School of Engineering and Technology

Lakshmangarh, Rajasthan

INDEX

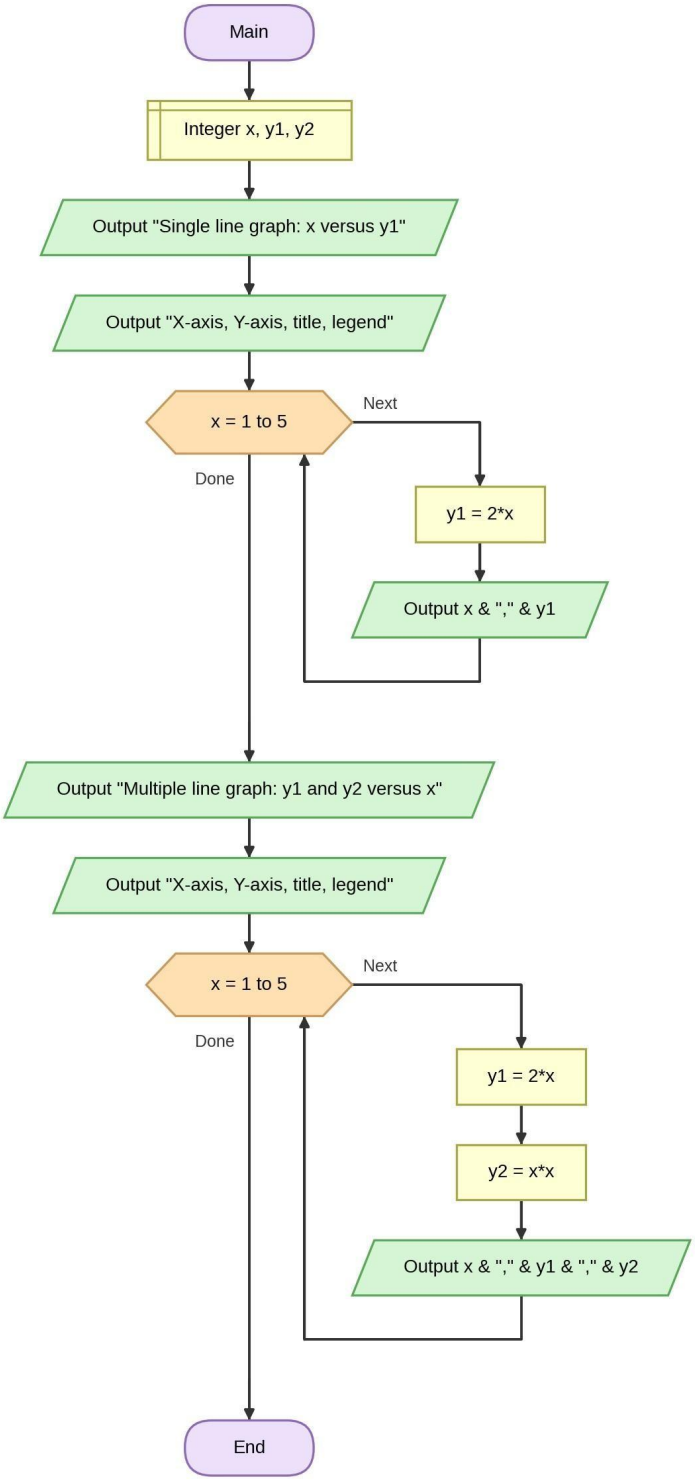
S. No.	Program	Link	Page
1	Graphs using Matplotlib	Program 1	3
2	Time Execution	Program 2	6
3	Memory Used	Program 3	8
4	Time Complexity	Program 4	10
5	Linear Search	Program 5	12
6	Largest Element	Program 6	14
7	Smallest Element	Program 7	16
8	Bubble Sort – Ascending Order	Program 8	18
9	Bubble Sort – Descending Order	Program 9	20
10	Insert Element at the End	Program 10	22
11	Insert Element at a Specified Position	Program 11	24
12	Delete Element from a Specified Position	Program 12	26
13	Delete Element by Value	Program 13	28
14	Factorial using Recursion	Program 14	30
15	Sum of Numbers using Recursion	Program 15	32
16	Fibonacci Series using Recursion	Program 16	34
17	Graph Representation using Classes and Objects	Program 17	35
18	Represent the Given Directed Graph (Q18)	Program 18	40
19	Represent the Given Directed Graph (Q19)	Program 19	45
20	Graph Creation Using User Input	Program 20	49

Program 1: Graphs using Matplotlib

Aim

To write a Python program using Matplotlib to plot and display single-line and multiple-line graphs with appropriate labels, title, and legend.

Flowchart



Program

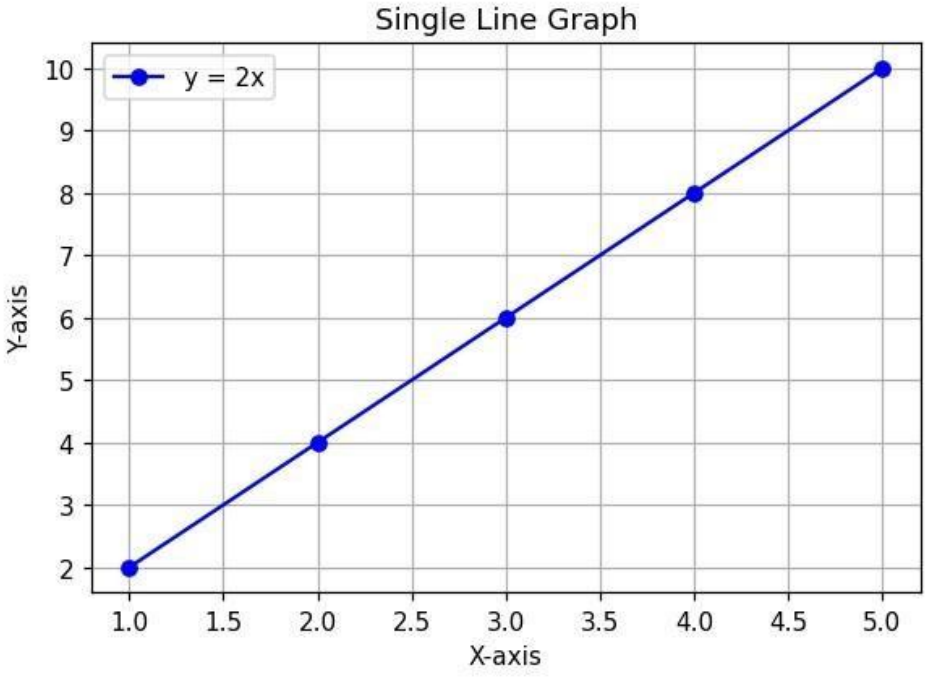
```
import matplotlib.pyplot as plt

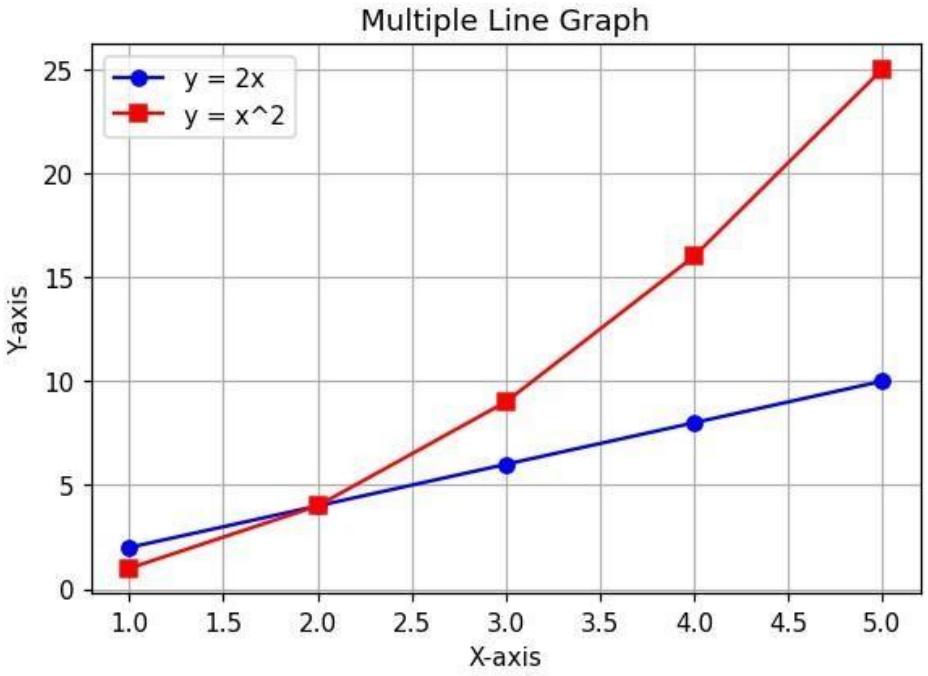
x = [1, 2, 3, 4, 5]
import y1 = [2, 4, 6, 8,
10]matplotlib.pyplot as plt y2 = [1,
4, 9, 16, 25]
x = [1, 2, 3, 4, 5]
#                   Single-line
graphplt.figure(figsize=(6,
4))y1 = [2, 4, 6, 8, 10]

plt.plot(x, y1, color="blue", marker="o", label="y = 2x")y2 = [1, 4, 9, 16, 25]
plt.xlabel("X-axis")
#plt.ylabel("Y-axis")---------- Single-line graph --------
plt.title("Single Line Graph")plt.figure(figsize=(6, 4))
plt.plot(x, y1, color="blue", plt.legend()
plt.grid(True)marker="o", label="y = 2x") plt.xlabel("X-axis")
plt.show()plt.ylabel("Y-axis") plt.title("Single Line Graph")
plt.legend() plt.grid(True) plt.show()
# Multiple-line graph
plt.figure(figsize=(6, 4))
#plt.plot(x, y1, color="blue", marker="o", label="y = 2x")---------- Multiple-line graph ---------
plt.plot(x, y2, color="red", marker="s", label="y = x^2")plt.figure(figsize=(6, 4))
plt.plot(x, y1, color="blue", plt.xlabel("X-axis")marker="o", label="y = 2x")
plt.plot(x, y2, color="red",
plt.ylabel("Y-axis")marker="s", label="y = x^2") plt.xlabel("X-axis")

plt.title("Multiple Line Graph")plt.legend()plt.ylabel("Y-axis")
plt.title("Multiple Line Graph")
plt.grid(True)plt.legend() plt.grid(True) plt.show()
plt.show()
```

Sample Output





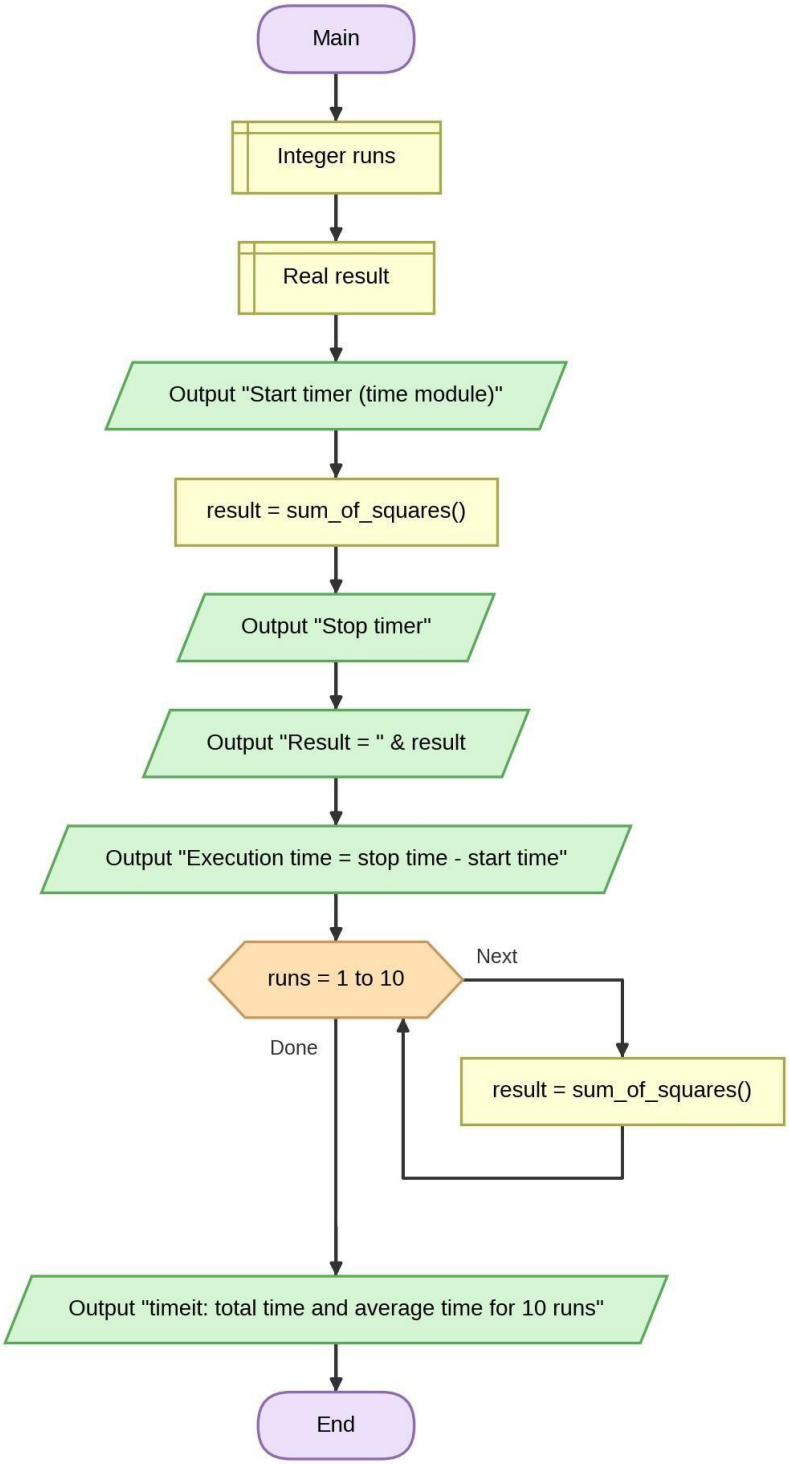
Back to Index

Program 2: Time Execution

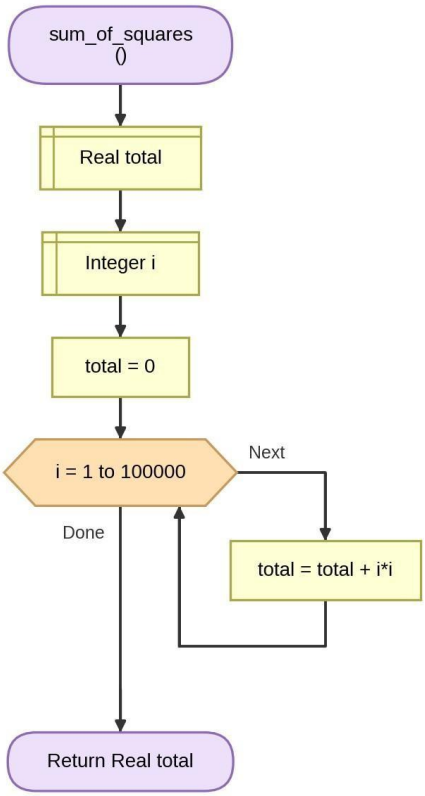
Aim

To write a Python program to calculate and display the execution time of a program using the time and timeit modules.

Flowchart



Main



Function sum_of_squares

Program

```
import time
 def sum_of_squares():
     total = 0 def     for i
 in                  range(1,
 100001):sum_of_squares():
         total += i * i
     total = 0     for i in
     return         total
 range(1, 100001):
 # Using timetotal += i * i
 start   =   time.time()return
 total
 result  =  sum_of_squares()
 end = time.time()
 # ---------- Using the time module ---------
 print("Result of function :", result)
 start = time.time() result = sum_of_squares()
 print("Execution time (time) : {:.6f} seconds".format(end - start))
 end = time.time() print("Result of
 # Using timeitfunction :", result) print("Executionruns = 10  time (time)
        : {:.6f} seconds".format(end - start))
 total_time = timeit.timeit(sum_of_squares, number=runs)
 # ---------- Using the timeit module ---------runs = 10 total_time =
 print("Total time for", runs,timeit.timeit(sum_of_squares, number=runs) print("Total
 time for", runs, "runs       "runs (timeit): {:.6f} seconds".format(total_time))(timeit):
 {:.6f} seconds".format(total_time)) print("Average time per run    :
 print("Average time per run : {:.6f} seconds".format(total_time / runs))
 {:.6f} seconds".format(total_time / runs))
```

Sample Output

```
Result of function          : 333338333350000
Execution time (time)       : 0.005291 seconds
  Total time for 10 runs (timeit): 0.049479 seconds
Average time per run        : 0.004948 seconds
```

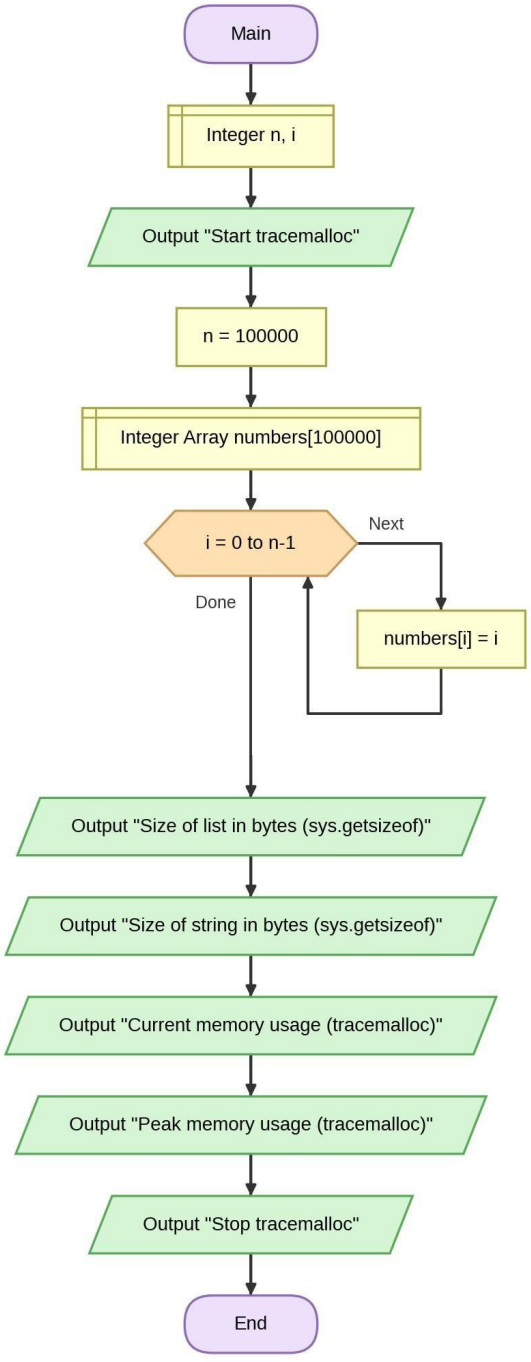
Back to Index

Program 3: Memory Used

Aim

To write a Python program to measure and display the memory occupied during program execution using the sys and tracemalloc modules.

Flowchart



Program

```
import sys import
tracemalloc.start()tracemalloc

#numbers = [i for i in range(100000)]Start tracing memory
allocations text = "Hello" * 1000tracemalloc.start()
numbersprint("Size of list object :", sys.getsizeof(numbers),
"bytes")= [i for i in range(100000)] print("Size of string
object :", sys.getsizeof(text), "bytes") text = "Hello" *
1000
current, peak = tracemalloc.get_traced_memory()
#print("Current memory usage : {:.2f} KB".format(current / 1024))-------
--- Using the sys module ---------print("Sizeprint("Peak memory
usage : {:.2f} KB".format(peak / 1024))of list object :",
sys.getsizeof(numbers), "bytes") print("Size of string object :",
sys.getsizeof(text), "bytes")
tracemalloc.stop()
# ---------- Using the tracemalloc module ---------current, peak
 = tracemalloc.get_traced_memory() print("Current memory usage :
{:.2f} KB".format(current / 1024)) print("Peak memory usage       :
{:.2f} KB".format(peak / 1024))

# Stop tracing
tracemalloc.stop()
```

Sample Output

```
Size of list object : 800984 bytes
Size of string object : 5041 bytes
Current memory usage : 3904.25 KB
Peak memory usage      : 3904.43 KB
```

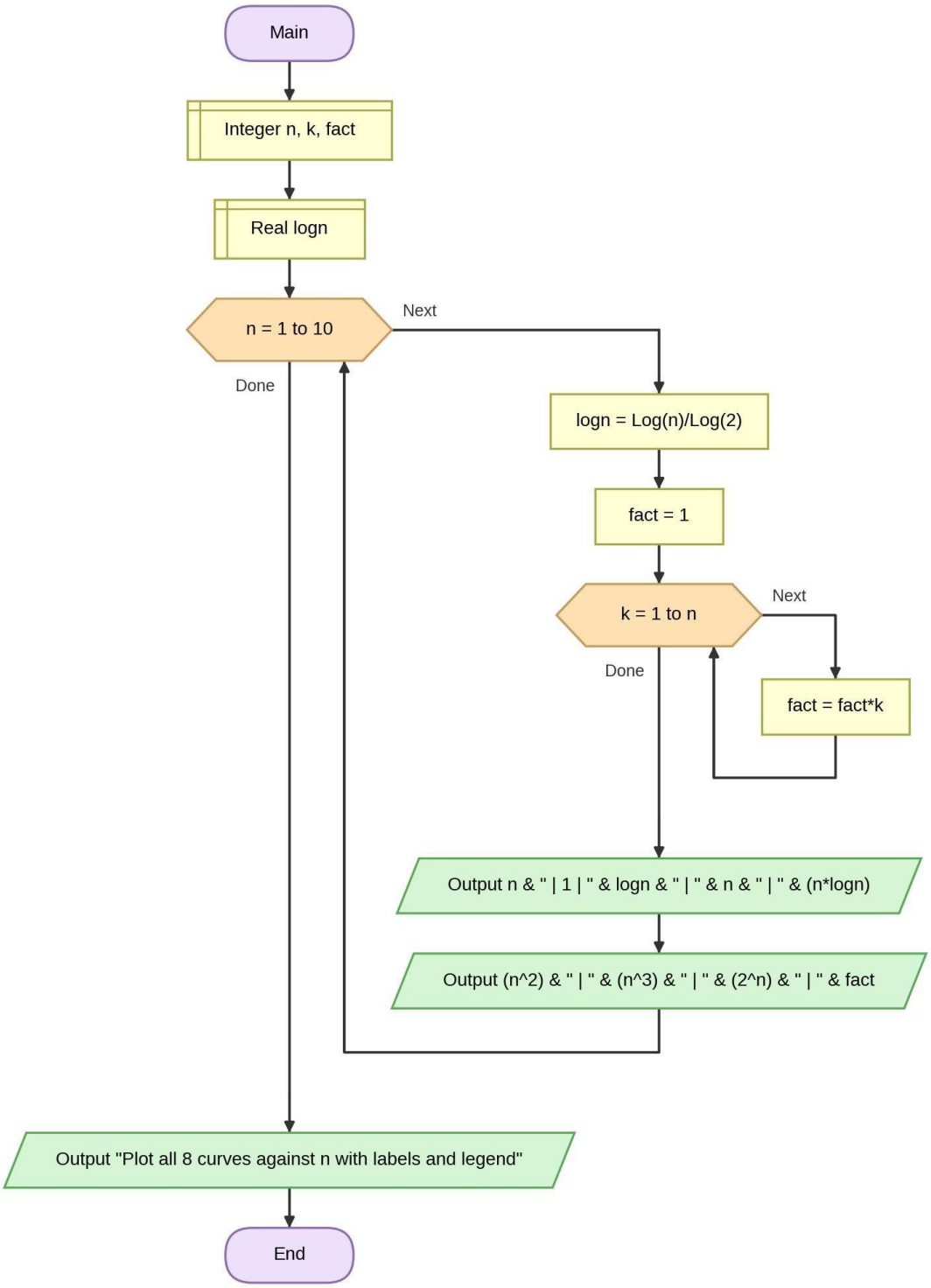
Back to Index

Program 4: Time Complexity

Aim

To write a Python program using Matplotlib to plot and compare different time complexities such as O(1), O(log n), O(n), O(n log n), $O(n^2)$, $O(n^3)$, $O(2^n)$, and O(n!).

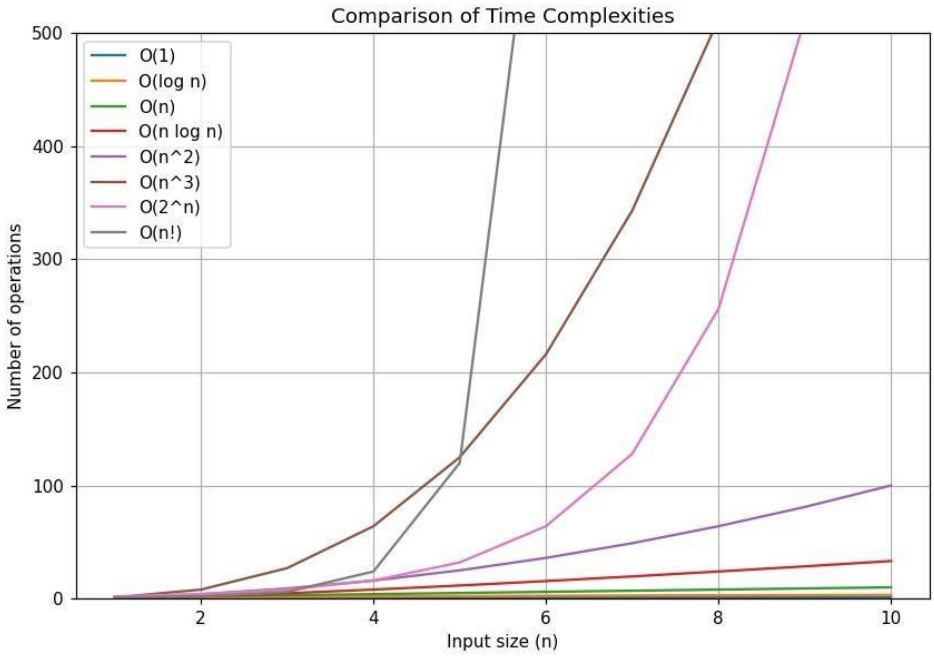
Flowchart



Program

```
import matplotlib.pyplot as pltmath import numpy as np import
matplotlib.pyplot as plt n = np.arange(1, 11)      # input
sizes 1 to
n = np.arange(1, 11)
10
o_1 = np.ones(len(n))
o_1o_logn = np.log2(n)= np.ones(len(n))        # O(1)
o_n = n
o_logn = np.log2(n)                            # O(log n)
o_nlogn = n * np.log2(n)
o_n2 = n ** 2o_n = n                           # O(n)
o_n3 = n ** 3o_nlogn = n * np.log2(n)          # O(n log n)
o_2n = 2.0 ** no_n2 = n ** 2                   # O(n^2)
o_fact = np.array([math.factorial(int(i)) for i in n], dtype=float)o_n3
= n ** 3      # O(n^3) o_2n = 2.0 ** n       # O(2^n)
o_factplt.figure(figsize=(9, 6))= np.array([math.factorial(int(i)) for i in n],
dtype=float) # O(n!) plt.plot(n, o_1, label="O(1)")
plt.plot(n, o_logn, label="O(log n)")
plt.figure(figsize=(9,plt.plot(n,    o_n,
label="O(n)")6))  plt.plot(n,  plt.plot(n,
o_nlogn,     label="O(n     log     n)")o_1,
label="O(1)")    plt.plot(n,    o_logn,
plt.plot(n,                          o_n2,
label="O(n^2)")label="O(log          n)")
plt.plot(n,    o_n,    plt.plot(n,    o_n3,
label="O(n^3)")label="O(n)")  plt.plot(n,
o_nlogn,         plt.plot(n,         o_2n,
label="O(2^n)")label="O(n    log    n)")
plt.plot(n,   o_n2,   plt.plot(n,   o_fact,
label="O(n!)")label="O(n^2)") plt.plot(n,
o_n3, label="O(n^3)") plt.plot(n, o_2n,
plt.ylim(0, 500)
plt.xlabel("Input size (n)")label="O(2^n)") plt.plot(n, o_fact,
plt.ylabel("Number of operations")label="O(n!)")
plt.title("Comparison of Time Complexities")
plt.ylim(0,plt.legend() 500)   # limit y-axis so slow curves stay plt.grid(True)visible
plt.xlabel("Input size (n)") plt.ylabel("Number of operations")
plt.show()plt.title("Comparison of Time Complexities") plt.legend() plt.grid(True)
plt.show()
```

Sample Output



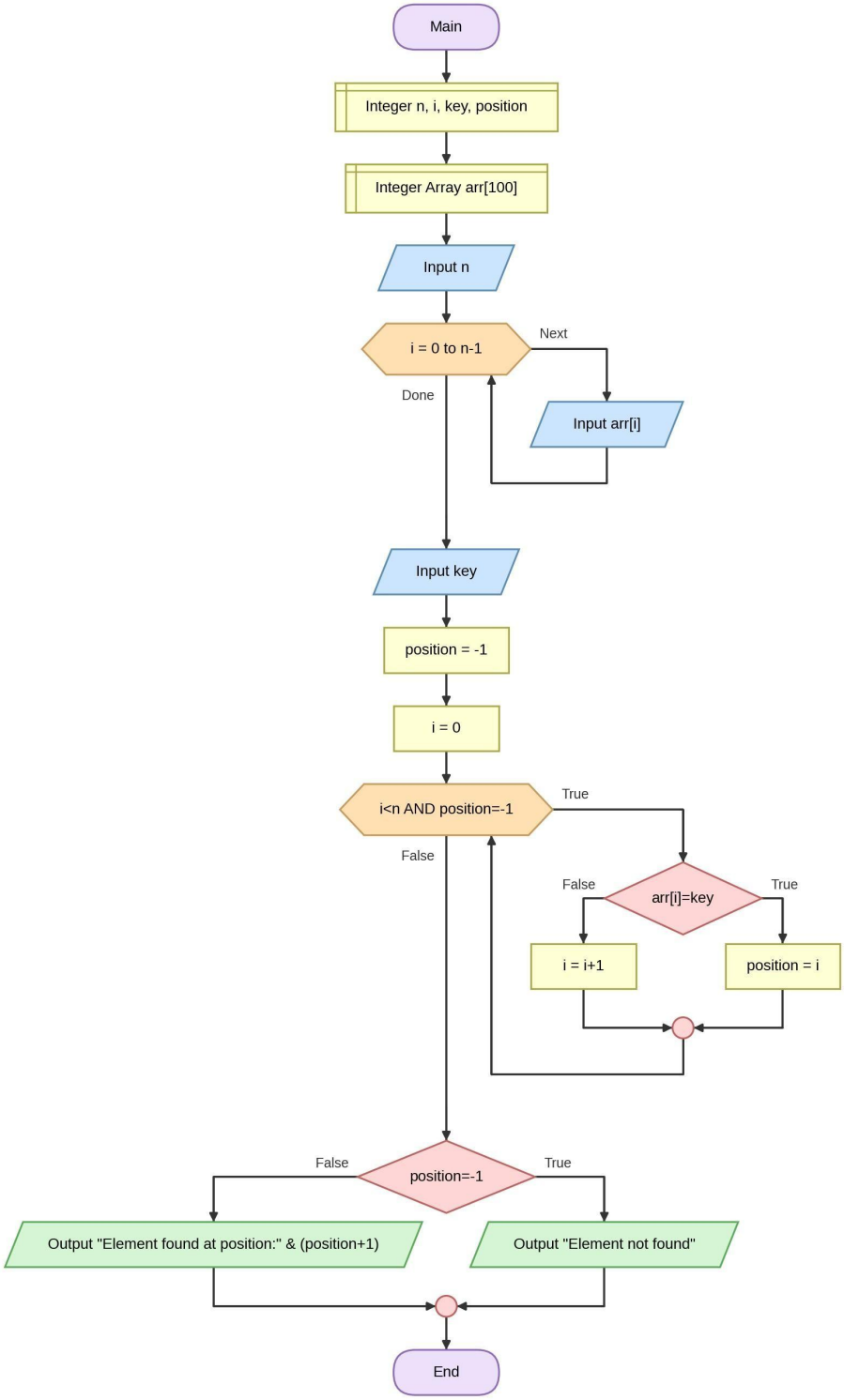
Back to Index

Program 5: Linear Search

Aim

To write a Python program to search for a given element in an array using Linear Search.

Flowchart



Program

```
arr = list(map(int, input("Enter array elements separated by space: ").split()))
for i in range(len(arr)):
key = int(input("Enter the element to search: "))
if arr[i] == key:
for i in range(len(arr)):print("Element", key, "found at position", i + 1, "(index",
str(i) + ")")     if arr[i] == key:break else: print("Element", key, "not found in
the array")
        print("Element", key, "found at position", i + 1,
"(index", i, ")")         break else:
    print("Element", key, "not found in the array")
```

Sample Output

```
Enter array elements separated by space: 10 25 30 45 60
Enter the element to search: 45
Element 45 found at position 4 (index 3)
```

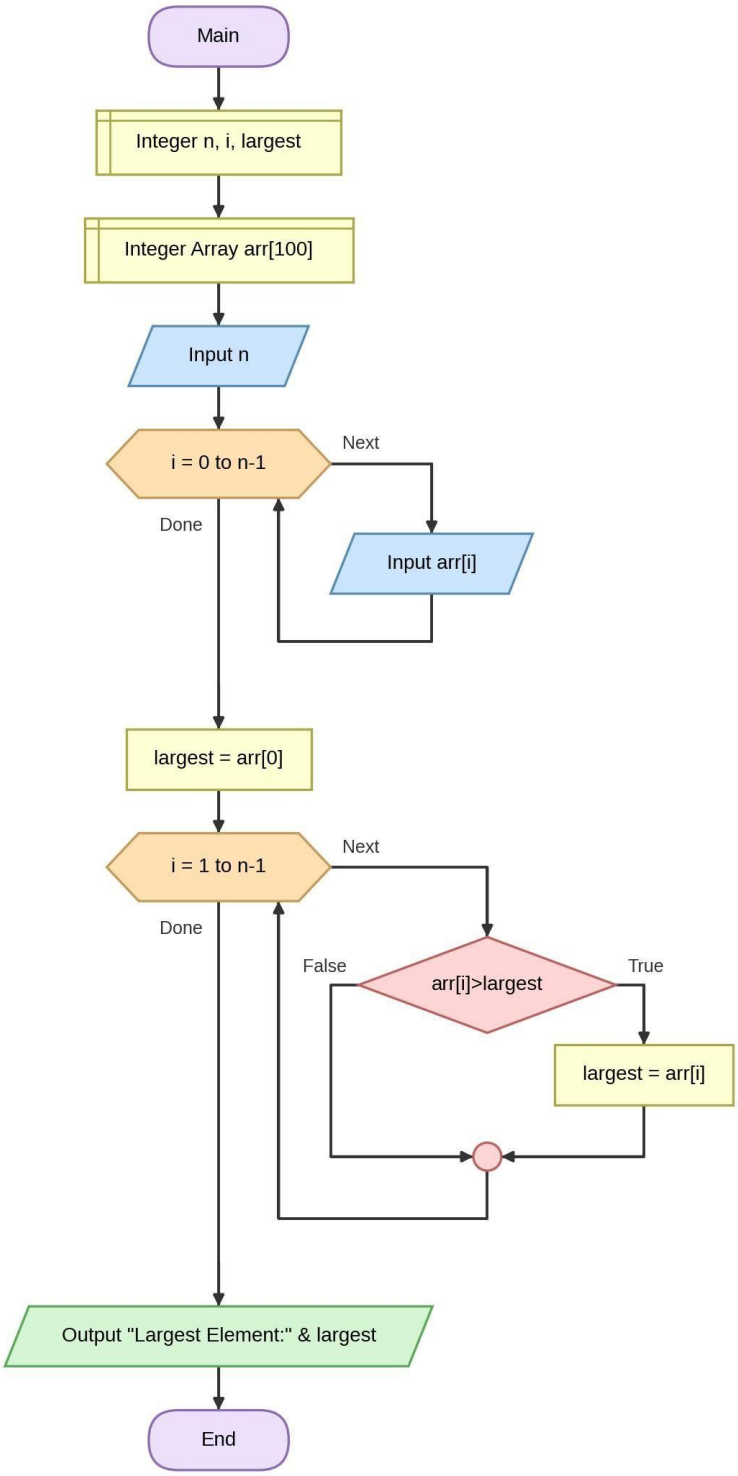
Back to Index

Program 6: Largest Element

Aim

To write a Python program to find the largest element in an array.

Flowchart



Program

```
arr = list(map(int, input("Enter array elements separated by space:
").split()))


For range(1,.split())len(arr)):
  if arr[i] > largest:
largest = arr[0]
largest = arr[i]
for i in range(1, len(arr)):
print("Array   if arr[i] > largest: :", arr)
largest = arr[i]print("Largest element :",
largest)
print("Array :", arr)
print("Largest element :",
largest)
```

Sample Output

```
Enter array elements separated by space: 12 45 7 89 23
Array            : [12, 45, 7, 89, 23]
Largest element : 89
```

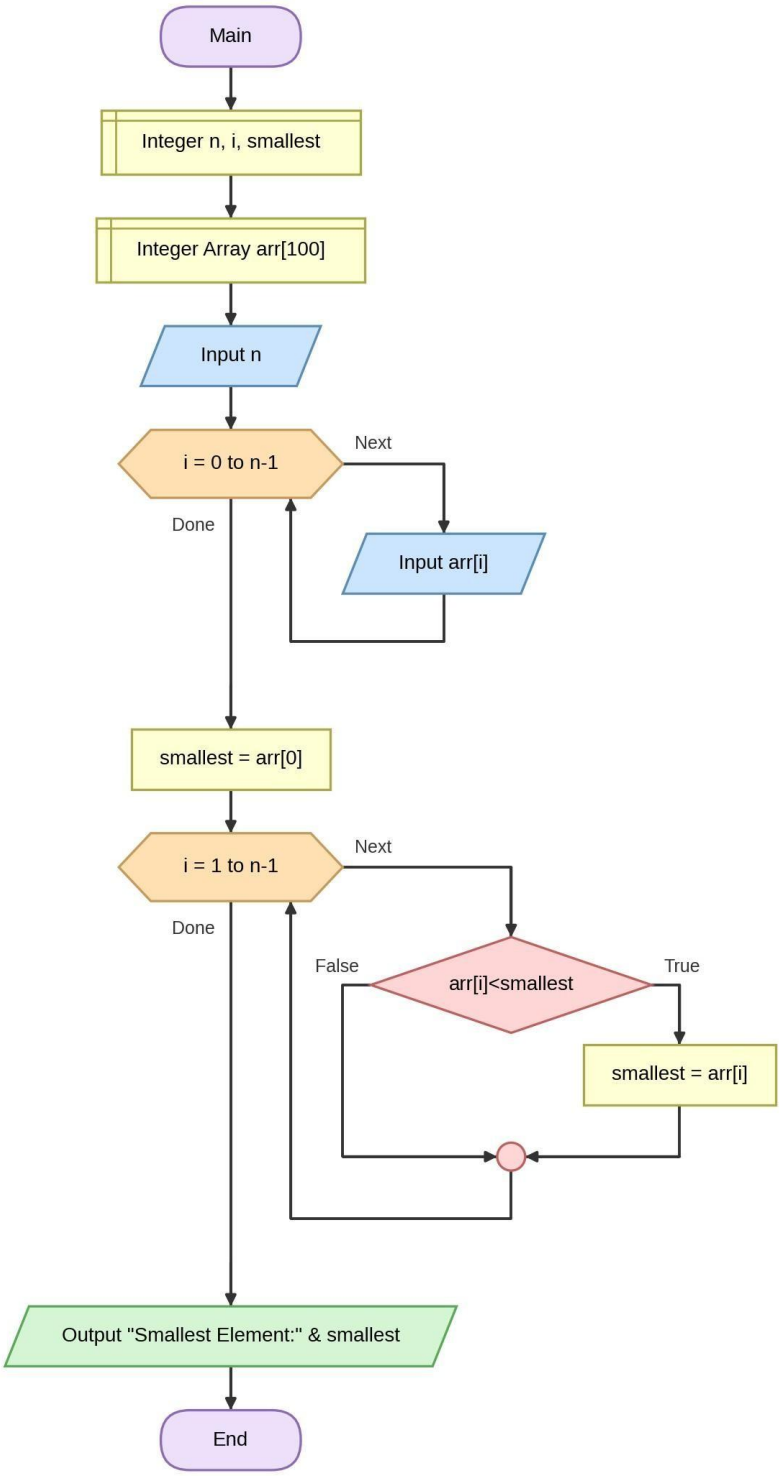
Back to Index

Program 7: Smallest Element

Aim

To write a Python program to find the smallest element in an array.

Flowchart



Program

```
arr = list(map(int, input("Enter array elements separated by space:
").split()))
)range(1,.split()))len(arr)):
  if arr[i] < smallest:
smallest = arr[0]
smallest = arr[i]
for i in range(1, len(arr)):
print("Array   if arr[i] < smallest: :", arr)
smallest = arr[i]print("Smallest element :",
smallest)
print("Array :", arr)
print("Smallest element :",
smallest)
```

Sample Output

```
Enter array elements separated by space: 12 45 7 89 23
Array              : [12, 45, 7, 89, 23]
Smallest element : 7
```

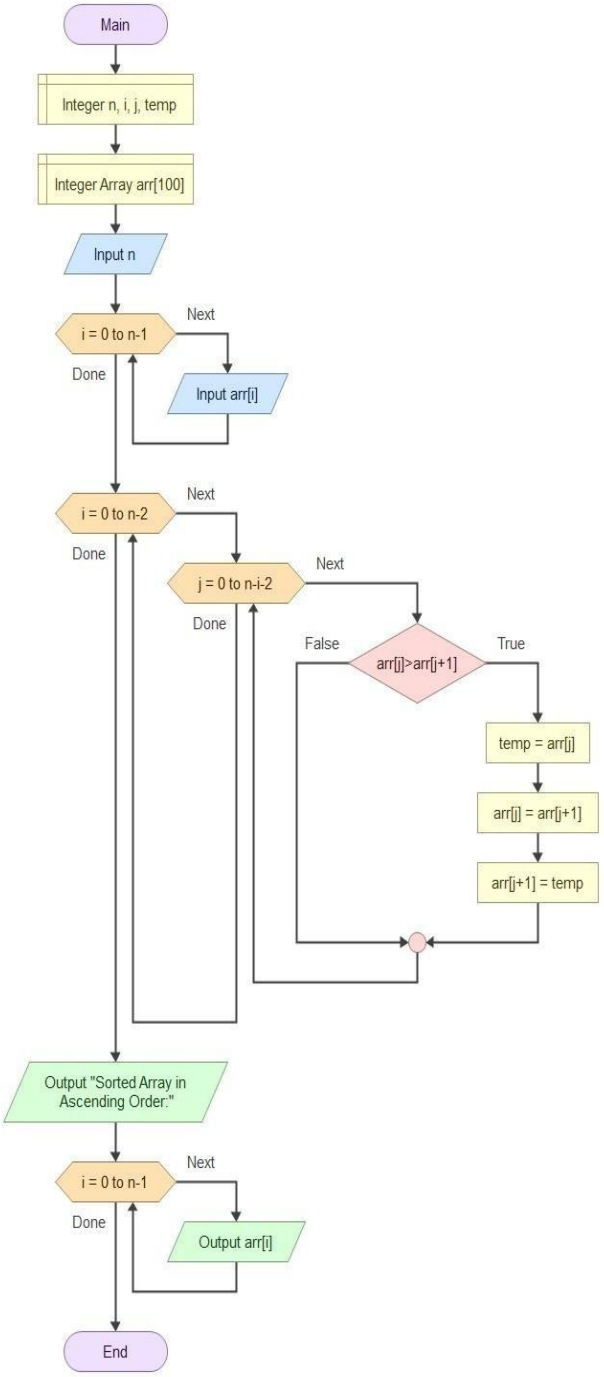
Back to Index

Program 8: Bubble Sort – Ascending Order

Aim

To write a Python program to sort an array in ascending order using Bubble Sort.

Flowchart




```
print("Original array :", arr)
n = len(arr) for i in range(n - 1):
print("Enter array elements separated by space: ").split())
```

```
  for j in range(n n =
len(arr)i - 1):
        if arr[j] > arr[j + 1]:                  # swap if out of
for i in range(n - 1):
   for j in range(n - i - 1):order arr[j], arr[j + 1] = arr[j + 1],
arr[j]        if arr[j] > arr[j + 1]:print("Sorted
(ascending) :", arr)
          arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted (ascending) :", arr)
```

Sample Output

```
Enter array elements separated by space: 64 34 25 12 22 11 90
Original array : [64, 34, 25, 12, 22, 11, 90]
Sorted (ascending) : [11, 12, 22, 25, 34, 64, 90]
```

Back to Index

Program 9: Bubble Sort – Descending Order

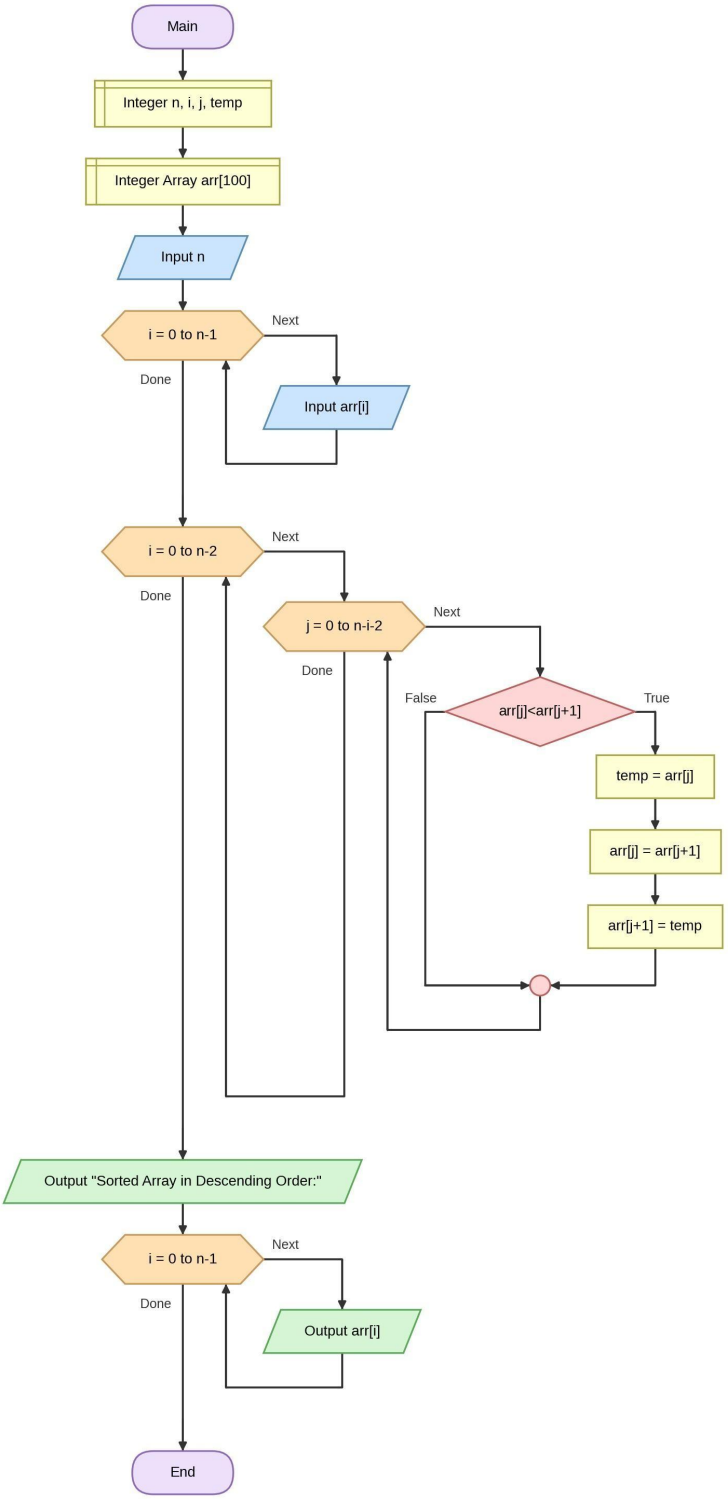
Aim

To write a Python program to sort an array in descending order using Bubble Sort.

Program

```
arr = list(map(int, input("Enter array elements separated by space:
").split()))
```

Flowchart`arr = list(map(int,`




```
input(
```

```
print("Original    "Enter array elements separated by space:
"array :", arr)
).split()))
```

```
n = len(arr) for i in range(n
print("Original array :", arr)
```

```
- 1): for j in range(n n =
len(arr)i - 1):

for i in range(n - 1):
    for j in range(n - i - 1):
      if arr[j] < arr[j + 1]:if arr[j] < arr[j + 1]:  # swap if out of
          arr[j], arr[j + 1] = arr[j + 1], arr[j]order     arr[j], arr[j +
1] = arr[j + 1], arr[j] print("Sorted (descending) :", arr)print("Sorted
(descending) :", arr)
```

Sample Output

```
Enter array elements separated by space: 64 34 25 12 22 11 90
Original array : [64, 34, 25, 12, 22, 11, 90]
Sorted (descending) : [90, 64, 34, 25, 22, 12, 11]
```

Back to Index

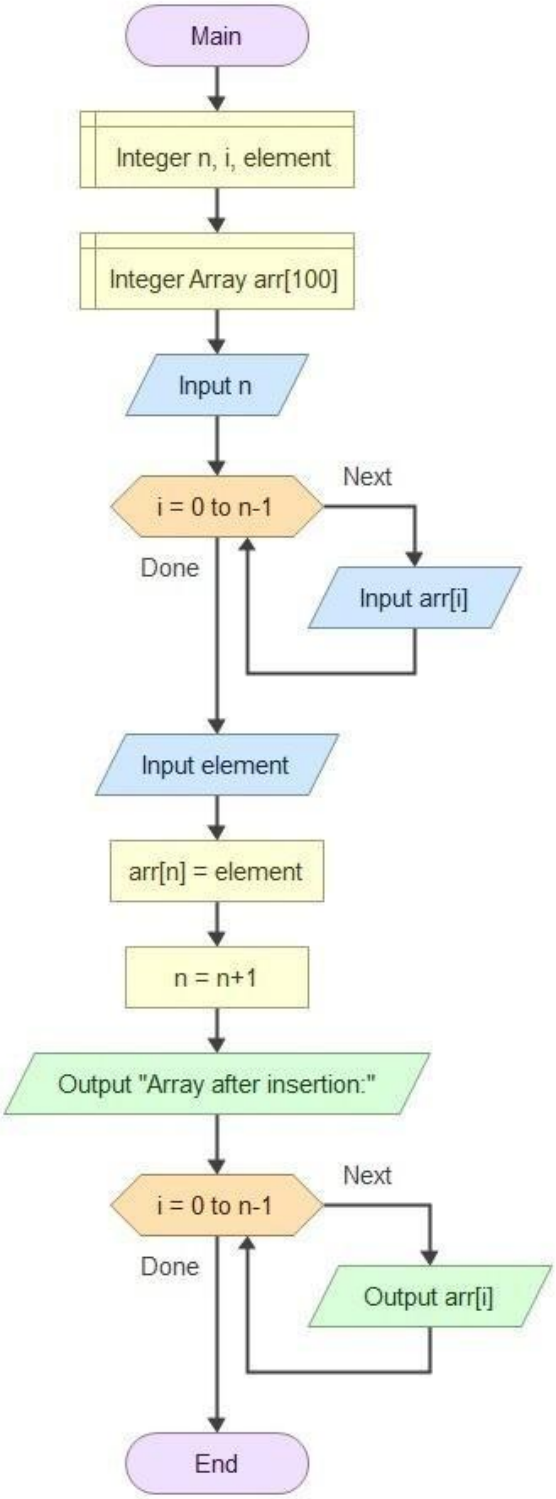
Program
```
arr = list(map(int, input("Enter array elements separated by space:
").split()))
```

Program 10: Insert Element at the End

Aim

To write a Python program to insert a new element at the end of an array.

Flowchart



Program `arr = list(map(int, input(element "Enter array elements separated by space: "= int(input("Enter the element to insert at the end: "))).split())) print("Array before insertion :", arr) arr.append(element) element = int(input("Enter the element to insert at the end: ")) print("Array after insertion :", arr)`

Program `arr = list(map(int, input("Enter array elements separated by space: ").split()))`

```
print("Array before insertion :", arr)
arr.append(element) print("Array after
insertion :", arr)
```

Sample Output

```
Enter array elements separated by space: 10 20 30 40
Enter the element to insert at the end: 50
Array before insertion : [10, 20, 30, 40]
Array after insertion : [10, 20, 30, 40, 50]
```

Back to Index

Program 11: Insert Element at a Specified Position

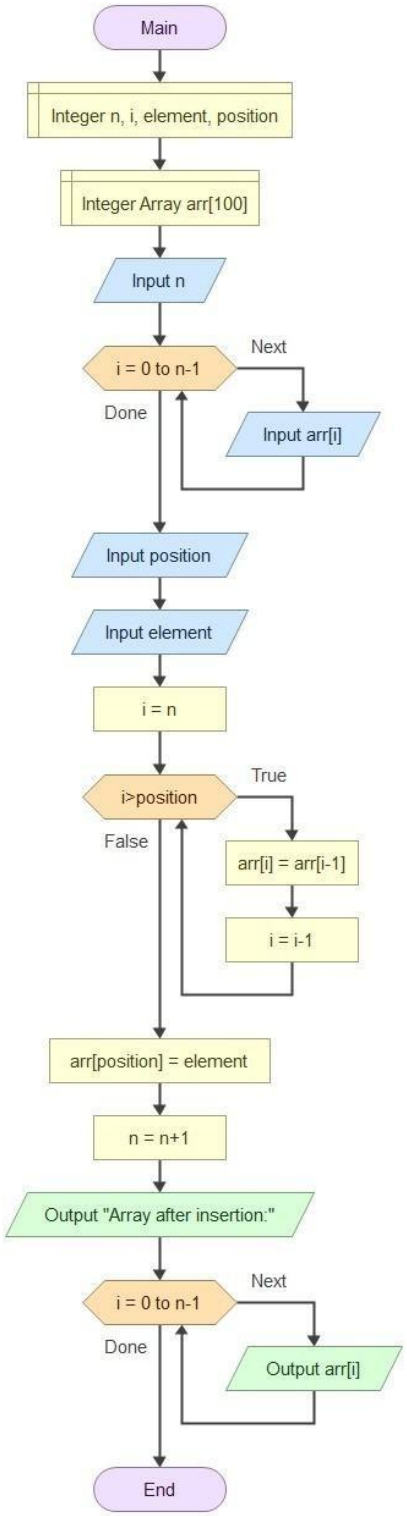
Aim

To write a Python program to insert a new element at a specified position in an array.

Program

```
arr = list(map(int, input("Enter array elements separated by space:
").split()))
```

Flowchart



Program

```
arr = list(map(int, input( position "Enter array elements separated by space: "= int(input("Enter the position (index 0 to " + str(len(arr)) + "): "))
)element.split()))= int(input("Enter the element to insert: ")) print("Array before insertion :", arr)
position = int(input(
```

```
    "Enter the position (index 0 to " + str(len(arr)) + "): "
if)) position < 0 or position > len(arr):
element = int(input("Enter the element to insert:
"))print("Invalid position!") else:
print("Array before insertion :", arr)
     arr.insert(position, element)
if position < 0 or position > len(arr):print("Array after
insertion :", arr)
    print("Invalid position!") else:
```

Sample `print("Array after insertion :", arr)` `arr.insert(position, element)` **Output**

```
Enter array elements separated by space: 10 20 30 40
Enter the position (index 0 to 4): 2
Enter the element to insert: 25
Array before insertion : [10, 20, 30, 40]
Array after insertion : [10, 20, 25, 30, 40]
```

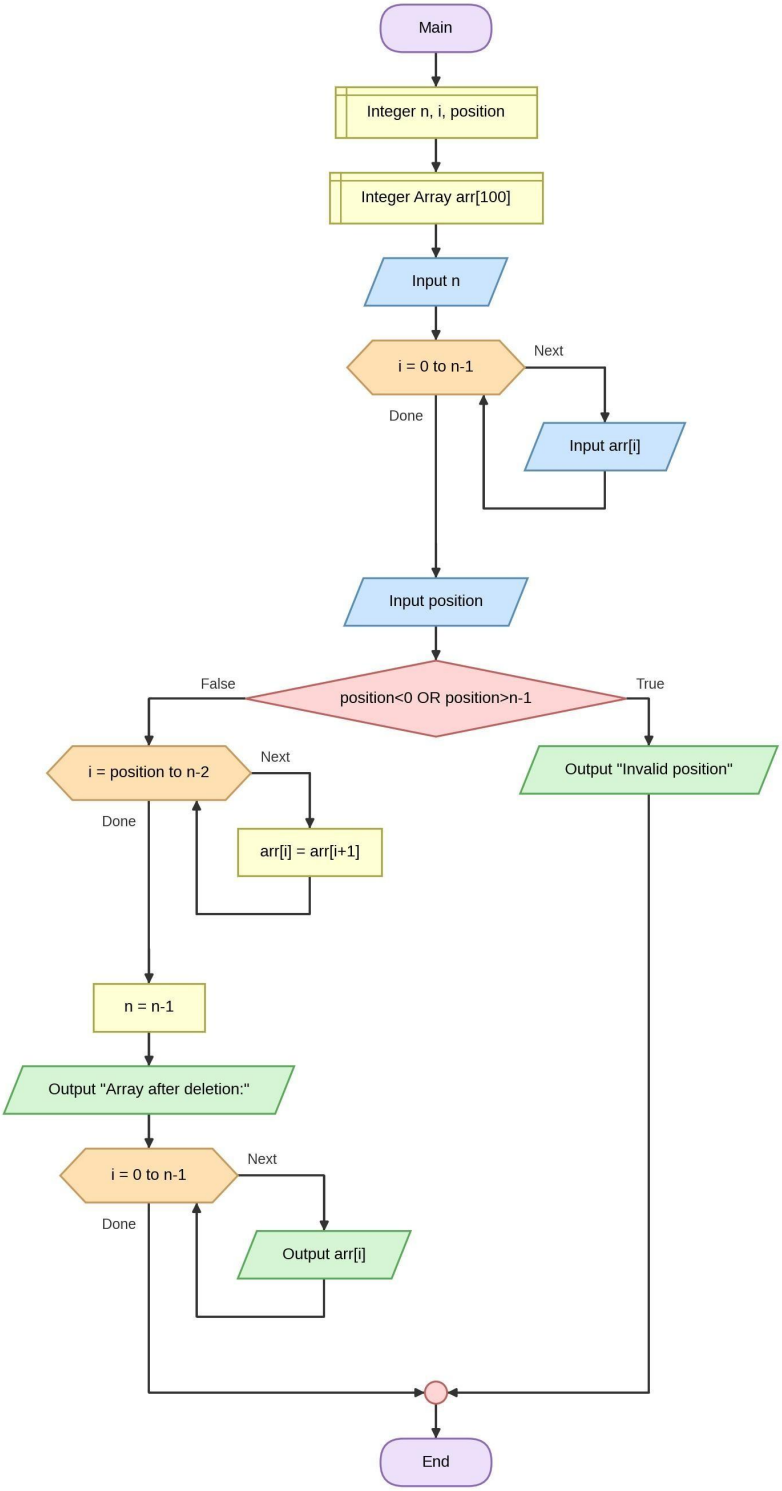
Back to Index

Program

```
arr = list(map(int, input("Enter array elements separated by space:
").split()))
```

12: Delete Element from a Specified Position

program to delete an element from a specified position in an array.



Program

```
arr = list(map(int, input(
    "Enter array elements separated by space: "
```

Program

~~Aim~~

To write a Python

Flowchart

```
arr).split= ()))list(map(int, input("Enter array elements separated by space: ").split()))
position = position = int(input(int(input("Enter the position to delete (index 0 to " +
str(len(arr) - 1) + "): "))
    "Enter the position to delete (index 0 to "print("Array before deletion :", arr)
    + str(len(arr) - 1) + "): "
if)) position < 0 or position >= len(arr):
     print("Invalid position!")
print("Array before deletion :", arr)
else:
if position < 0 or position >= len(arr):removed = arr.pop(position)
    print("Invalid position!")print("Deleted element     :",
removed) else:print("Array after deletion :", arr)
    removed = arr.pop(position)
print("Deleted element :", removed)
```

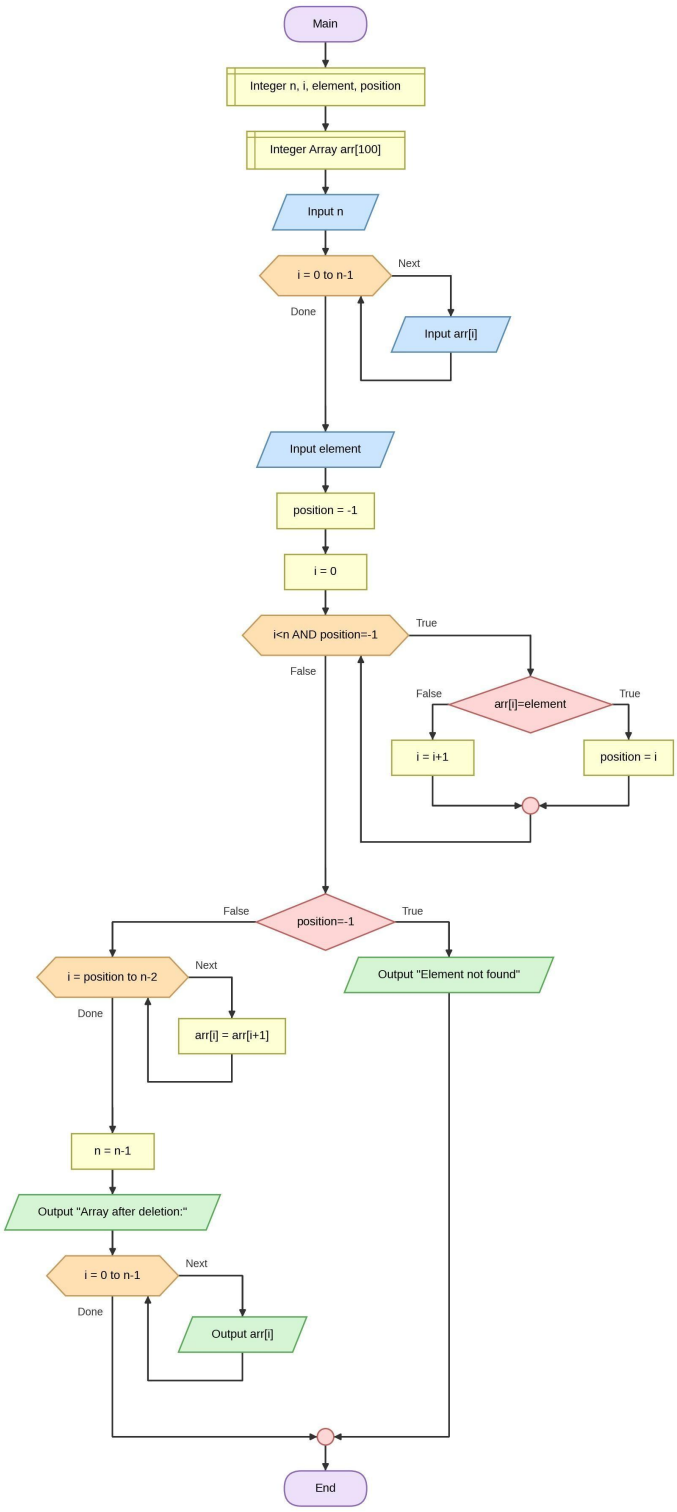
Sample print("Array after deletion :", arr)Output

```
Enter array elements separated by space: 10 20 30 40 50
Enter the position to delete (index 0 to 4): 2
Array before deletion : [10, 20, 30, 40, 50]
Deleted element         : 30
Array after deletion : [10, 20, 40, 50]
```

Back to Index

13: Delete Element by Value

program to delete a given value an array by searching for from first it.



Program

```
arr = list(map(int,
input(     "Enter array
elements separated by
space: " ).split()))

value = int(input("Enter the value to delete: "))

print("Array before deletion :", arr)
```

Program

~~Aim~~

To write a Python

Flowchart

```
for i in range(len(arr)):     if arr[i] == value:
        del arr[i]
        print("Value", value, "found at position", i + 1,
"and deleted")
```

Program

```
print("Array after deletion :", arr)        break

arr else:= list(map(int, input("Enter array elements separated by space: ").split()))
    print("Value", value, "not found in the array")
value = int(input("Enter the value to delete: "))
print("Array before deletion :", arr)

for i in range(len(arr)):
    if arr[i] == value:                    # value found at index i
del arr[i]    print("Value", value, "found at position", i + 1, "and
deleted")     print("Array after deletion :", arr) break else:
      print("Value", value, "not found in the array")
```

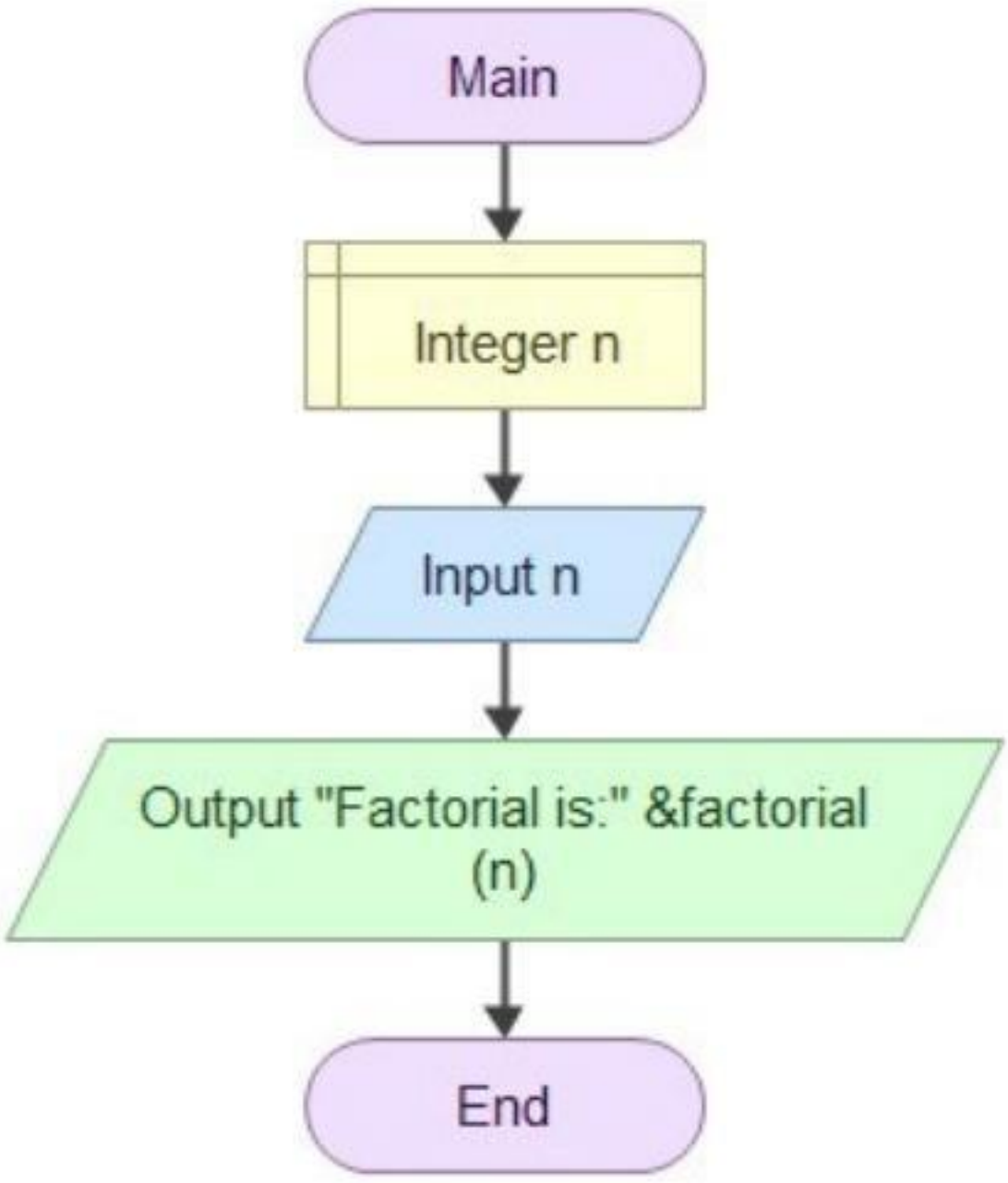
Sample Output

```
Enter array elements separated by space: 10 20 30 20 50
Enter the value to delete: 30
Array before deletion : [10, 20, 30, 20, 50]
Value 30 found at position 3 and deleted
Array after deletion : [10, 20, 20, 50]
```

Back to Index

14: Factorial using Recursion

program to find the factorial of a given number using recursion.



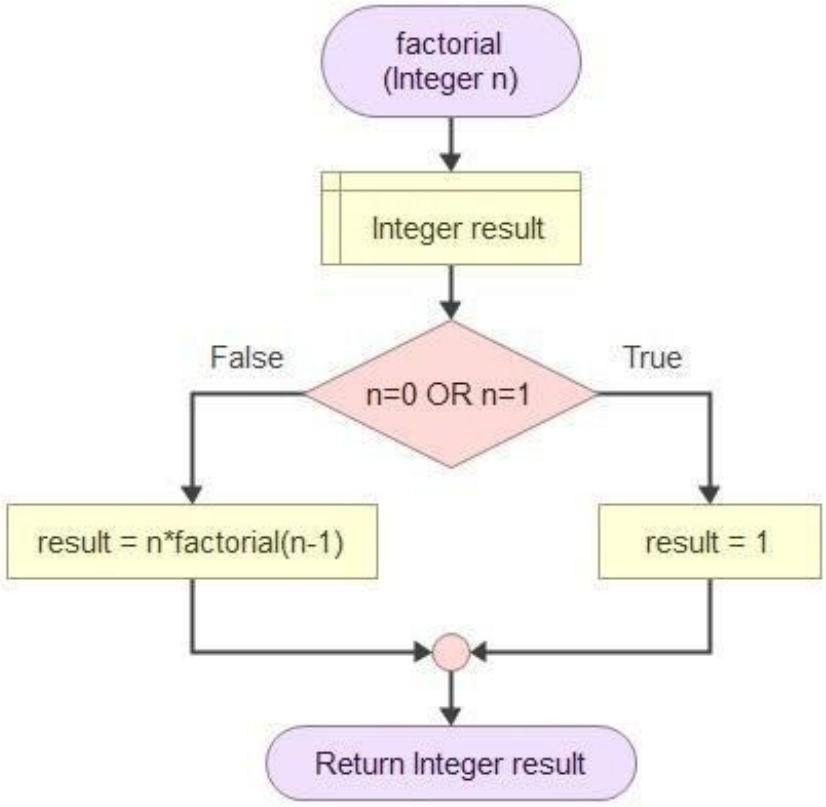
Main

Program

Aim

To write a Python

Flowchart



Function factorial

Program

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    # base case
    return 1
    return n * factorial(n - 1)
    return n * factorial(n - 1)   #
    recursive case

n = int(input("Enter a number: "))
if n < 0:
    print("Factorial is not defined for negative numbers")
else:
    print("Factorial of", n, "is", factorial(n))
```

Sample Output

```
Enter a number: 5
Factorial of 5 is 120
```

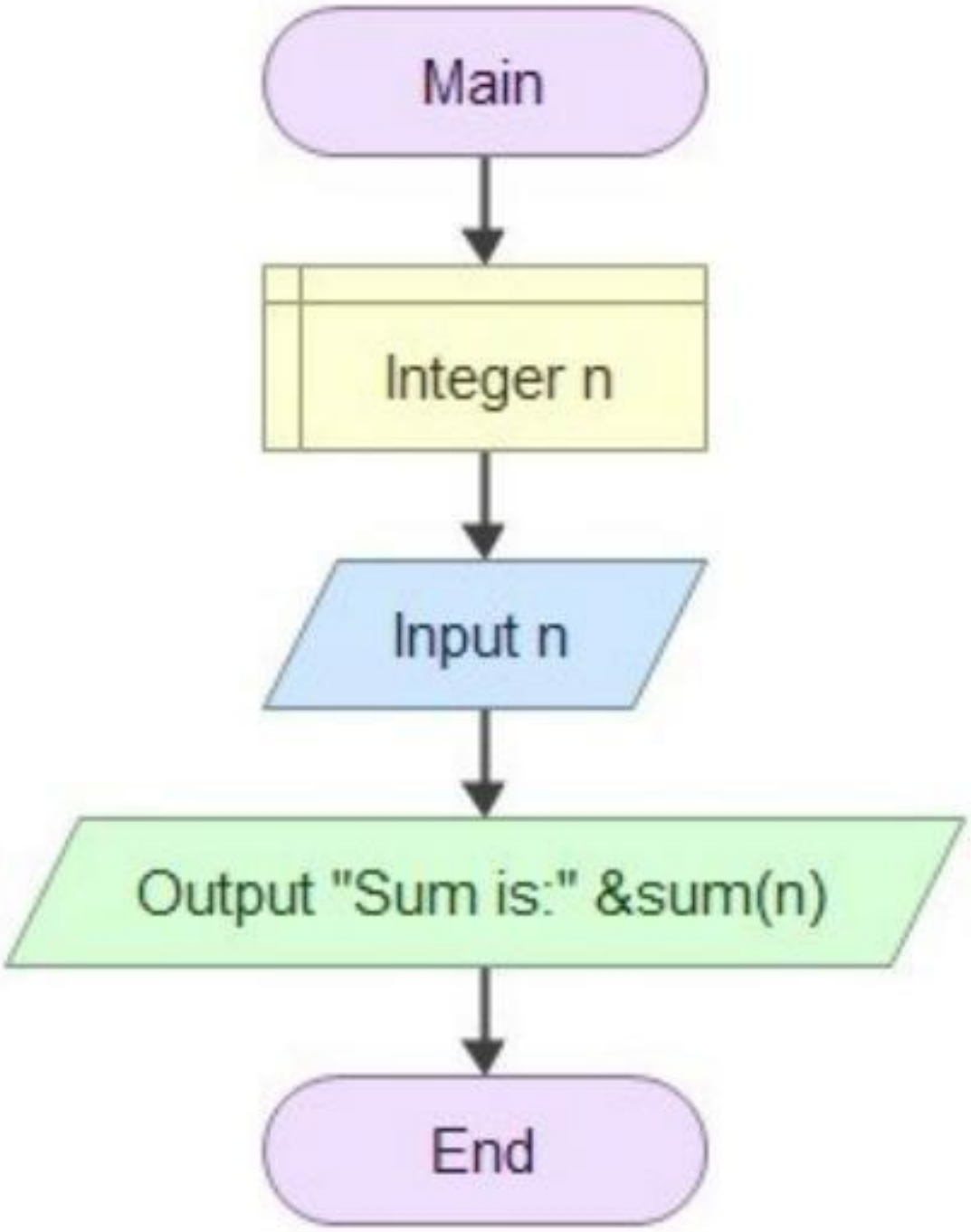
Back to Index

Program 15: Sum of Numbers using Recursion

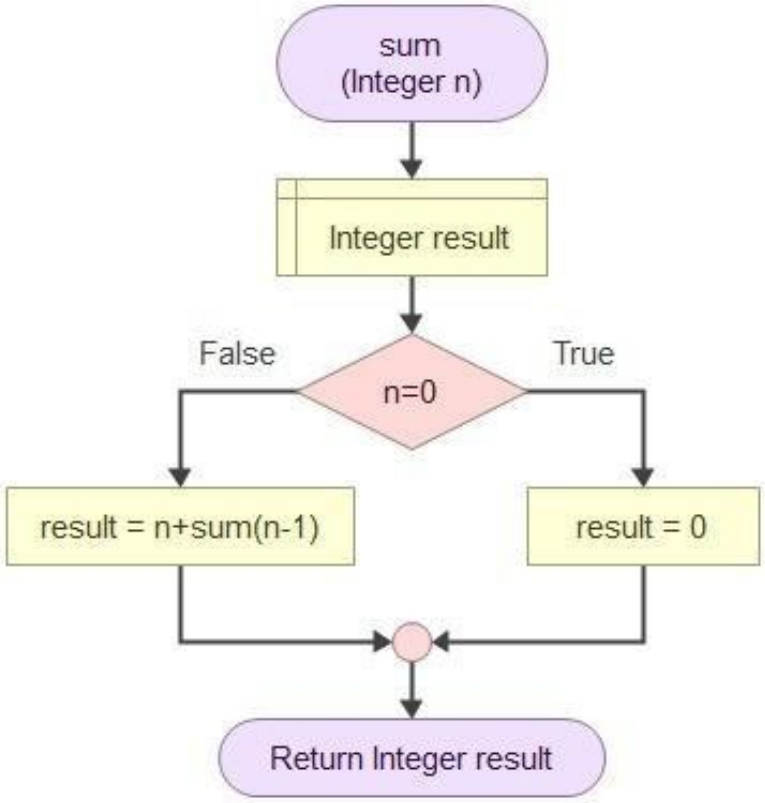
Aim

To write a Python program to find the sum of natural numbers up to a given number using recursion.

Flowchart



Main



Function sum

Program

```
def sum_numbers(n): def     if
 n == 0:sum_numbers(n): if n == 0:
 return 0
# base case           return 0
     return n + sum_numbers(n - 1)
      return n + sum_numbers(n - 1)    # recursive case
 n   =   int(input("Enter   a   number:   "))   nif   n   <   0:=
 int(input("Enter a number: "))      print("Please enter a
 non-negative number") ifelse:n < 0:     print("Sum of numbers
 from 1 to", n, "is", sum_numbers(n))print("Please enter a
 non-negative number") else:
 print("Sum of numbers from 1 to", n, "is", sum_numbers(n))
```

Sample Output

```
Enter a number: 10
Sum of numbers from 1 to 10 is 55
```

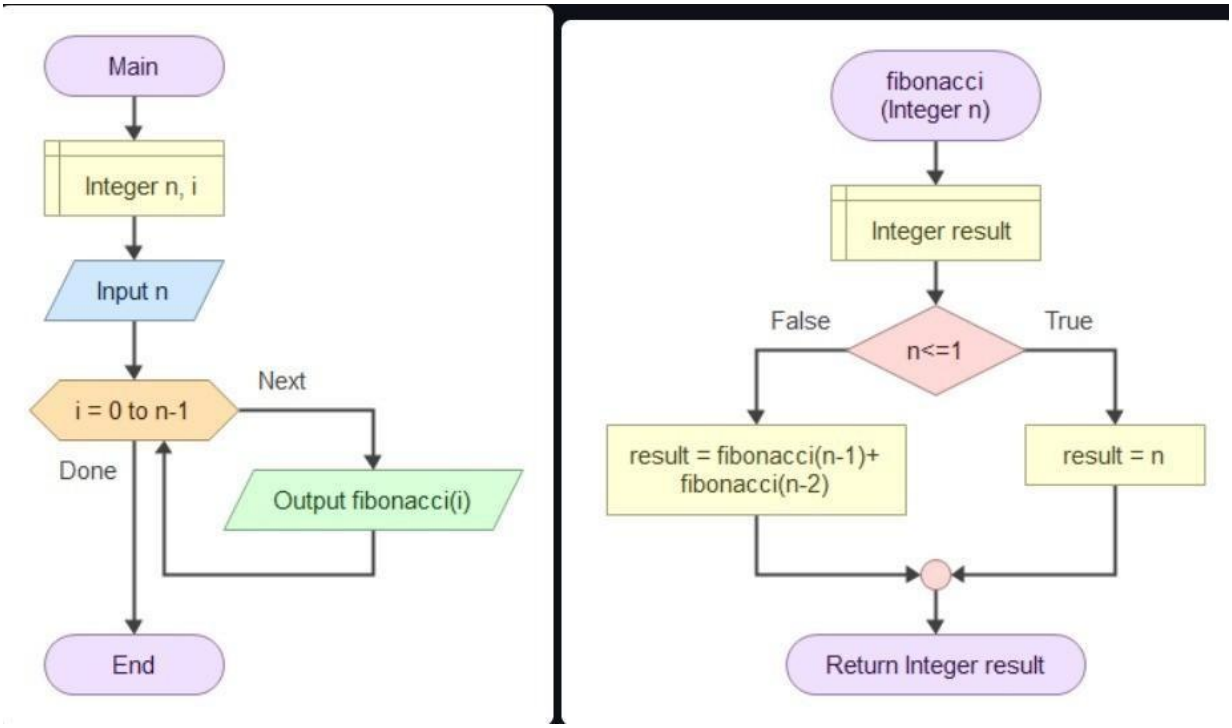
Back to Index

Program 16: Fibonacci Series using Recursion

Aim

To write a Python program to generate the Fibonacci series using recursion.

Flowchart



Program

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)

terms = int(input("Enter the number of terms: "))
print("Fibonacci series:")
for i in range(terms):
    print(fibonacci(i), end=" ")
print()
```

Sample Output

```
Enter the number of terms: 10
Fibonacci series:
0 1 1 2 3 5 8 13 21 34
```

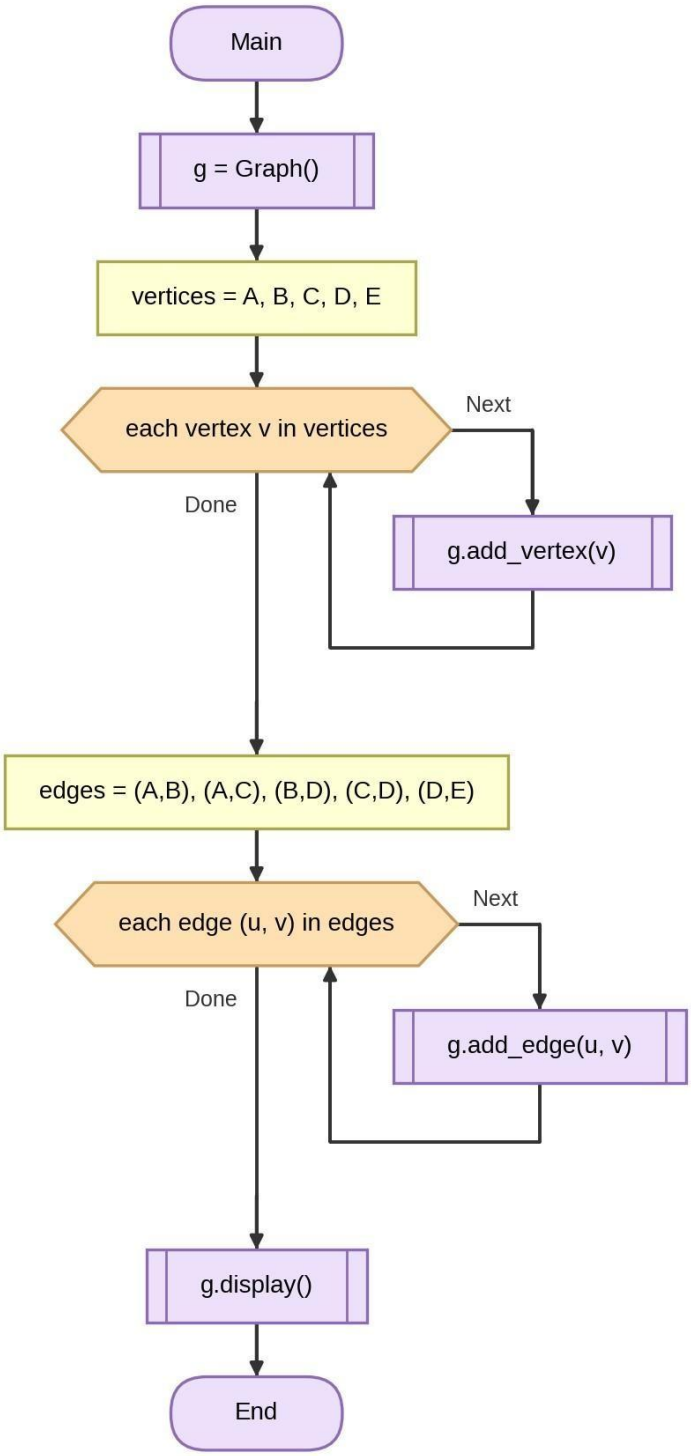
Back to Index

Program 17: Graph Representation using Classes and Objects

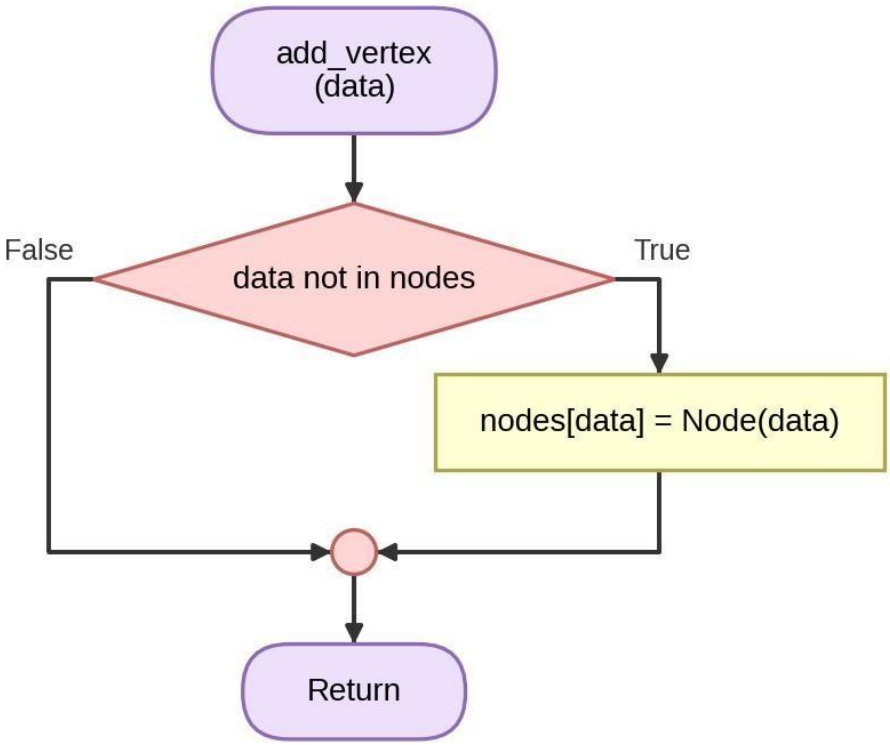
Aim

To write a Python program to represent a graph using Node and Graph classes and objects, add the given vertices and edges to the graph, and display the graph using an adjacency list.

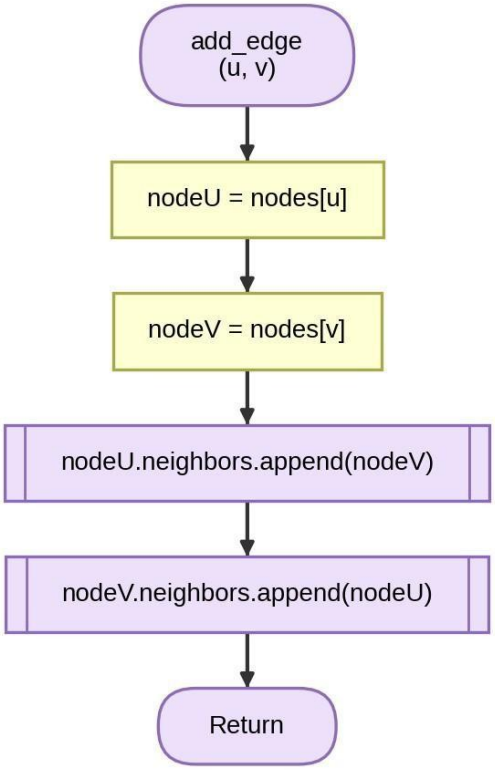
Flowchart



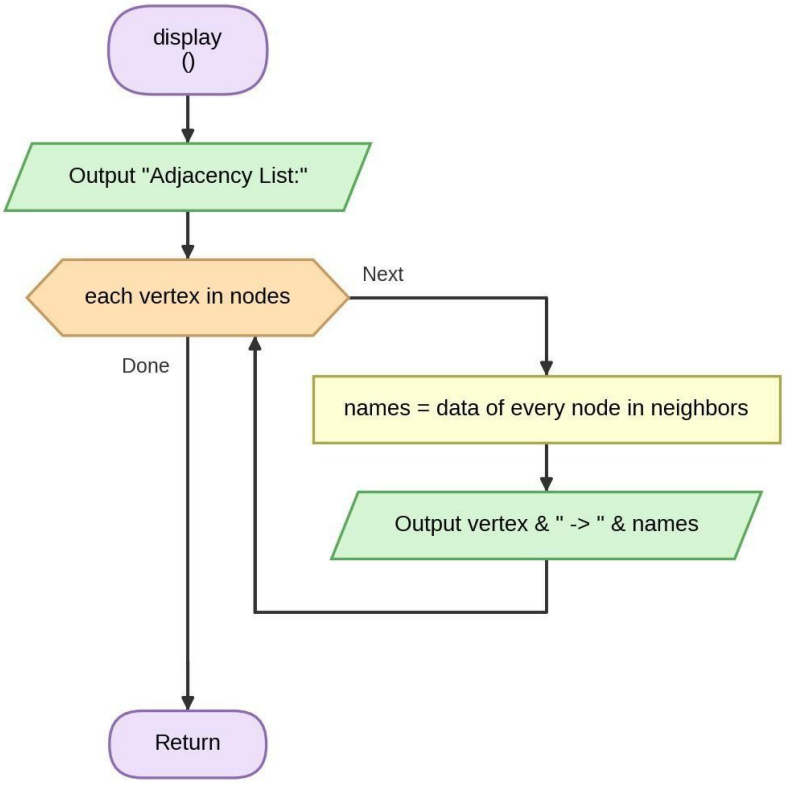
Main



Function add_vertex



Function add_edge (undirected)



Function display

Program

```
class Node: class
def           __init__(self,
data):Node:           def
self.data       =       data
__init__(self, data):
       self.neighbors      =      []
self.data = data
class Graph:self.neighbors = []
    def __init__(self):
self.nodes = {}                                  # list of adjacent Node objects
class Graph:
    def add_vertex(self, data):
        if data not in self.nodes:def __init__(self):     self.nodes = {}
        #            self.nodes[data] = Node(data)vertex name -> Node object


    def add_edge(self, u, v):def add_vertex(self, data):
        self.nodes[u].neighbors.append(self.nodes[v])if data not in self.nodes:
        self.nodes[v].neighbors.append(self.nodes[u]) self.nodes[data]
= Node(data)
    def display(self):
      print("Adjacency List:")def add_edge(self, u, v):# undirected edge u -- v
        for data, node in
self.nodes.items():self.nodes[u].neighbors.append(self.nodes[v])
            names = ", ".join(str(n.data) for n in
node.neighbors)self.nodes[v].neighbors.append(self.nodes[u])
            print(data, "->", names)
     def display(self):
g = Graph()
         print("Adjacency List:")          for
vertices = ["A", "B", "C", "D", "E"]data, node in self.nodes.items():
edges = [       names = ", ".join(str(n.data) for n in node.neighbors)
    ("A", "B"),print(data, "->", names)
    ("A", "C"),
    ("B", "D"),
    ("C", "D"),
g = Graph()
```

```
# Change the vertices and edges below to match the given graph
vertices = ["A", "B", "C", "D", "E"]
edges = [("A", "B"), ("A", "C"), ("B", "D"), ("C", "D"), ("D", "E")]

for v in vertices:

    ("D", "E")
]     g.add_vertex(v)


for v in vertices:for u, v in edges:
    g.add_vertex(v)
     g.add_edge(u, v)
for u, v in edges:
    g.add_edge(u, v)
g.display()
g.display()
```

Sample Output

```
Adjacency List:
A -> B, C
B -> A, D
C -> A, D
D -> B, C, E
E -> D
```

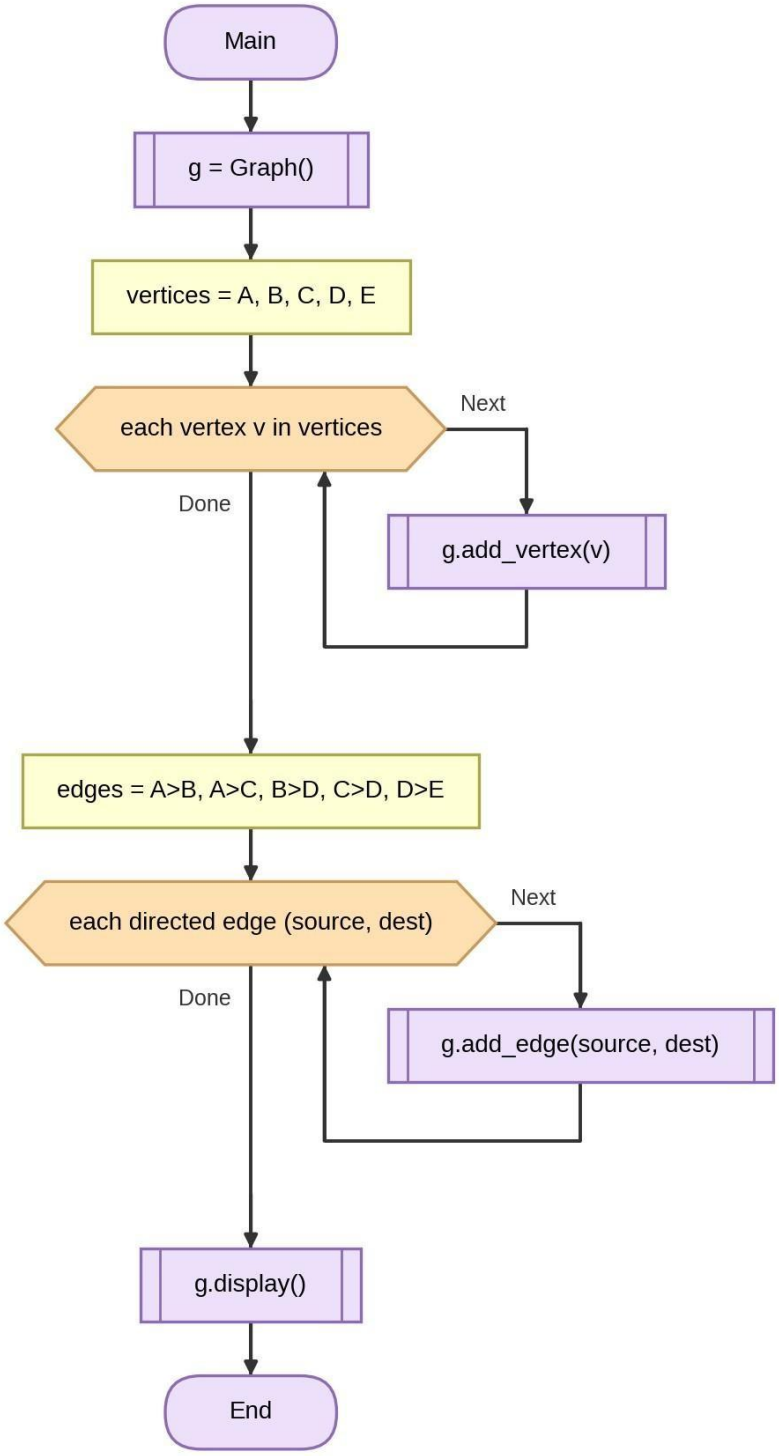
Back to Index

Program 18: Represent the Given Directed Graph (Q18)

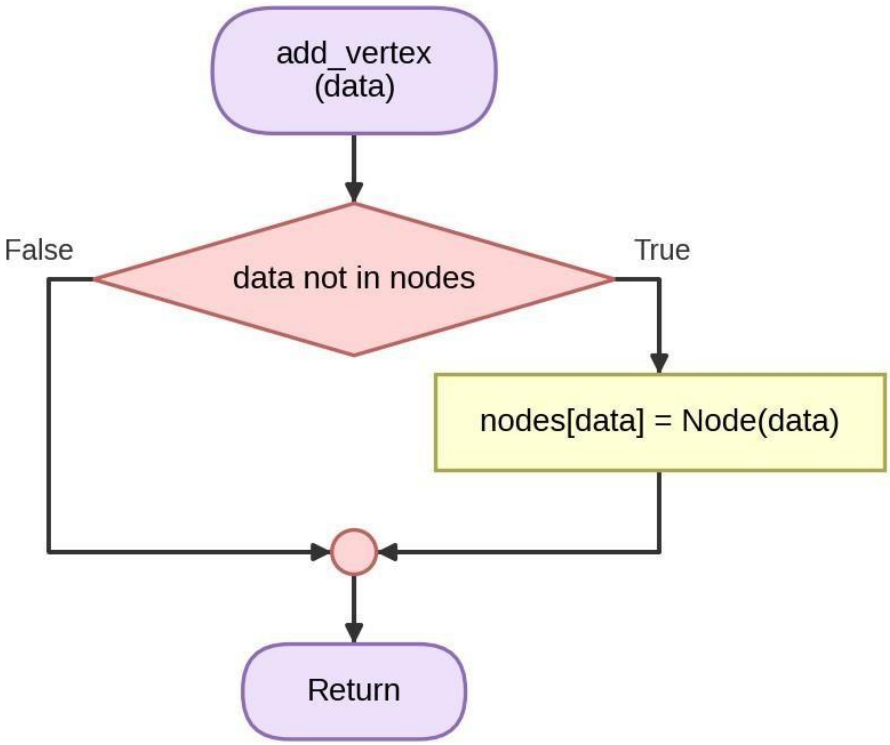
Aim

To write a Python program using classes and objects to represent the given directed graph, add all the vertices and directed edges, and display its adjacency list.

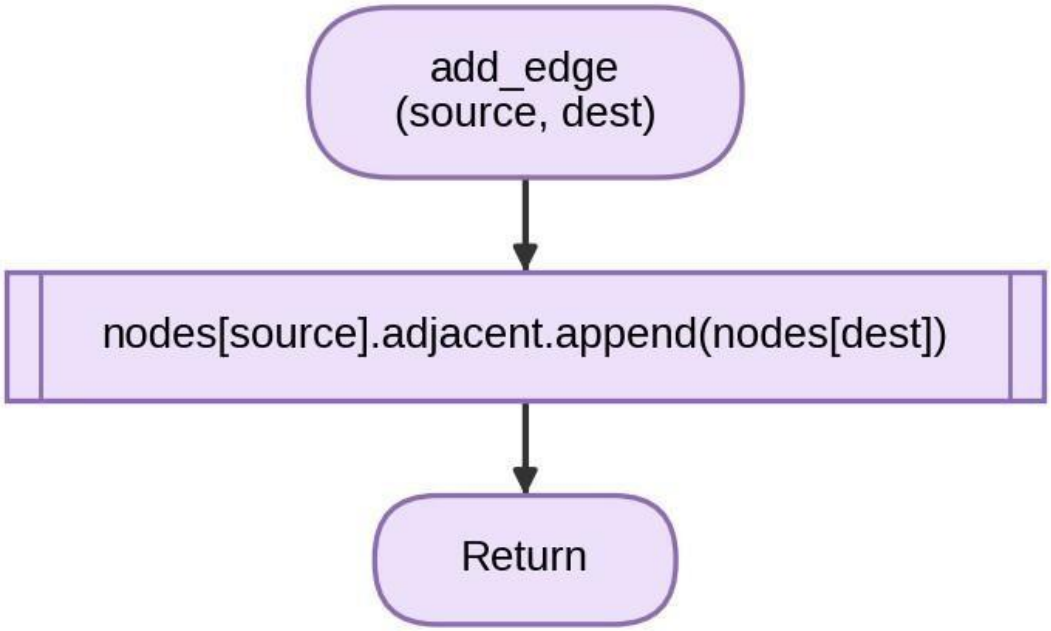
Flowchart


Main
g = Graph()
vertices = A, B, C, D, E
each vertex v in vertices
Next
Done
g.add_vertex(v)
edges = A>B, A>C, B>D, C>D, D>E
each directed edge (source, dest)
Next
Done
g.add_edge(source, dest)
g.display()
End

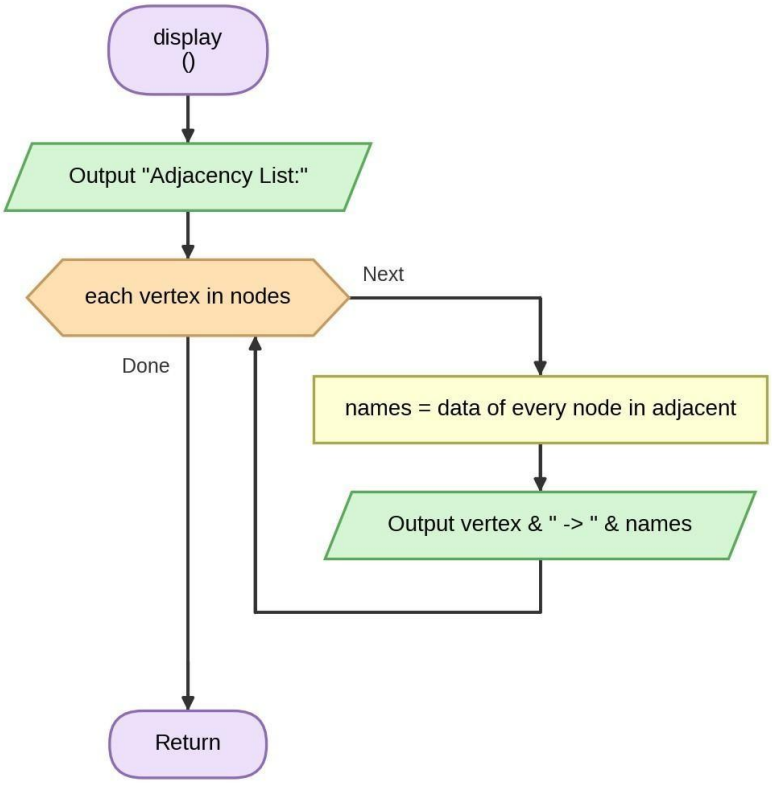
Main



Function add_vertex



Function add_edge (directed)



Function display

Program

```
class Node: class
def __init__(self,
data):Node: def
self.data = data
__init__(self, data):
    self.adjacent = []
self.data = data
class Graph:self.adjacent = []
    def __init__(self):
self.nodes = {}                                  # nodes this vertex points to
class Graph:
    def add_vertex(self, data):
if data not in self.nodes:def
__init__(self):
self.nodes[data] =
Node(data)self.nodes = {}

    def add_edge(self, source, dest):def add_vertex(self, data):
        self.nodes[source].adjacent.append(self.nodes[dest])
if data not in self.nodes:
    def display(self):self.nodes[data] = Node(data)
        print("Adjacency List:")
    for data, node in self.nodes.items():def add_edge(self, source, dest):      # directed edge source ->
dest
            names = ", ".join(str(n.data) for n in node.adjacent) or
"None"self.nodes[source].adjacent.append(self.nodes[dest])
            print(data, "->", names)
g = Graph()def display(self):
        print("Adjacency List:")          for
vertices = ["A", "B", "C", "D", "E"]data, node in self.nodes.items():
edges = [       names = ", ".join(str(n.data) for n in node.adjacent) or "None"
    ("A", "B"),print(data, "->", names)
    ("A", "C"),
    ("B", "D"),
```

```
    ("C", "D"),
g    ("D",
"E")= Graph()
]
# Change the vertices and directed edges below to match the given graph
for v in vertices:
vertices = ["A", "B", "C", "D", "E"]
edges = [("A", "B"), ("A", "C"), ("B", "D"), ("C", "D"), ("D", "E")]

for v in vertices:
    g.add_vertex(v) for

source, dest in edges:

   g.add_vertex(v)

for        source,        dest        in
edges:g.add_edge(source,      dest)
g.display()    g.add_edge(source, dest)

g.display()
```

Sample Output

```
Adjacency List:
A -> B, C
B -> D
C -> D
D -> E
E -> None
```

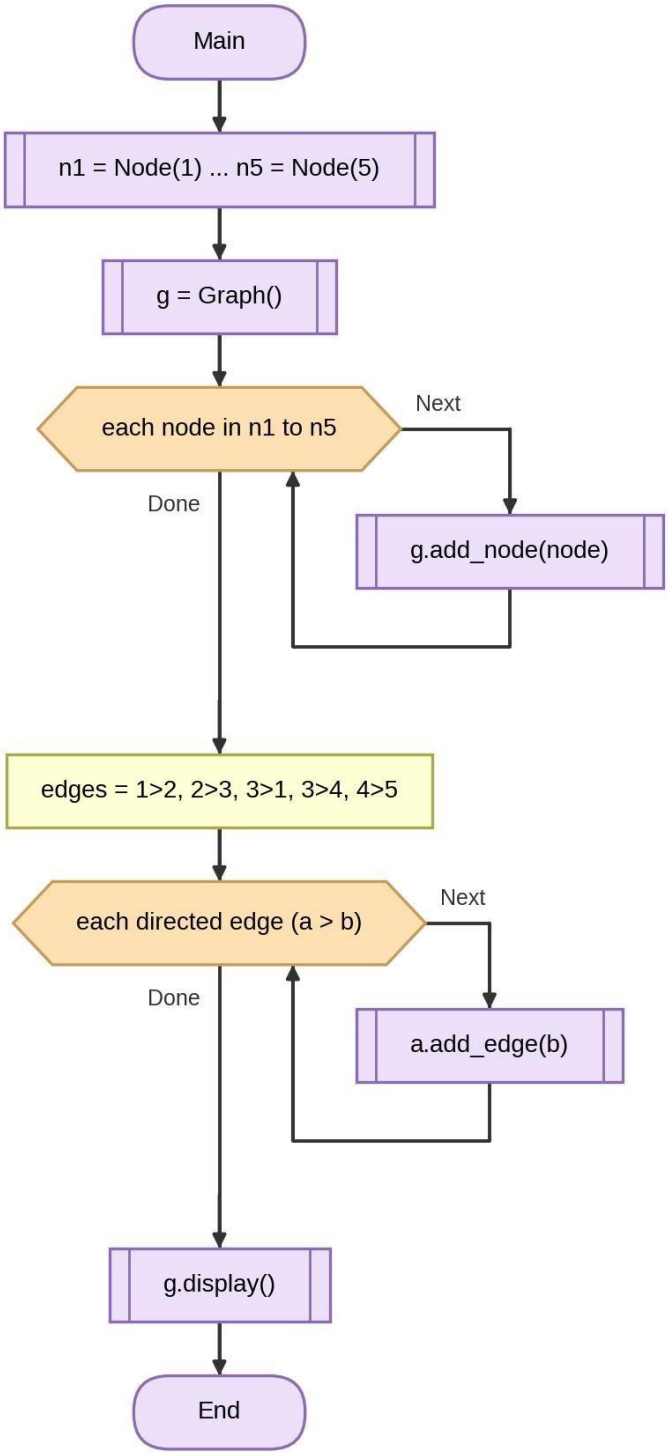
Back to Index

Program 19: Represent the Given Directed Graph (Q19)

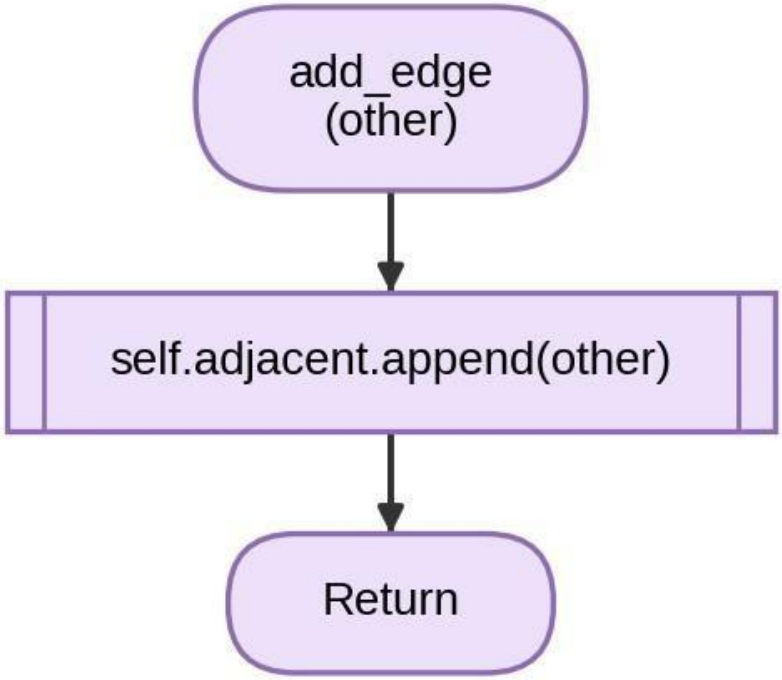
Aim

To write a Python program using classes and objects to represent the given directed graph by creating nodes for all vertices, adding the directed edges, and displaying the graph using an adjacency list.

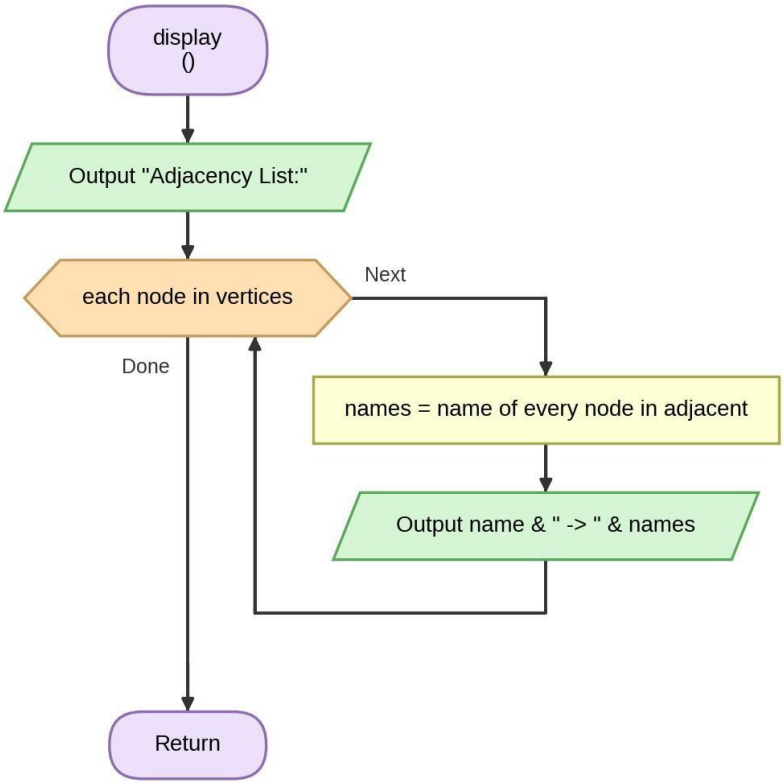
Flowchart


Main
n1 = Node(1) ... n5 = Node(5)
g = Graph()
each node in n1 to n5
Next
Done
g.add_node(node)
edges = 1>2, 2>3, 3>1, 3>4, 4>5
each directed edge (a > b)
Next
Done
a.add_edge(b)
g.display()
End

Main



Function Node.add_edge



Function display

Program

```
class Node:
```

```
class           def   __init__(self,
name):Node:    def
self.name   =   name__init__(self,
name):
        self.adjacent     =     []
self.name = name
    def add_edge(self, other):self.adjacent = []
        self.adjacent.append(other)
     def add_edge(self, other):            # directed edge self -> other
class Graph:self.adjacent.append(other)
    def __init__(self):
self.vertices = [] class
Graph:def
    def   add_node(self,   node):
self.vertices.append(node)__init
__(self): self.vertices = []
    def           display(self):
print("Adjacency      List:")def
add_node(self,          node):
for           node            in
self.vertices:self.vertices.ap
pend(node)
           names   =   ",   ".join(str(n.name)   for   n   in   node.adjacent)   or   "None"
print(node.name, "->", names)def display(self):

n1 = Node(1)print("Adjacency List:")
n2 = Node(2)for node in
self.vertices:
n3 = Node(3) names = ", ".join(str(n.name) for n in node.adjacent) or "None"
n4 = Node(4)print(node.name, "->", names) n5 = Node(5)

#g = Graph()Create nodes for all vertices (change to match the given
graph) n1 = Node(1) n2 = Node(2) n3 = Node(3) n4 = Node(4) n5 =
Node(5)
for node in (n1, n2, n3, n4, n5):
    g.add_node(node)
g = Graph() for node in (n1, n2,
n1.add_edge(n2)n3, n4, n5):
n2.add_edge(n3)g.add_node(node)
n3.add_edge(n1)
#n3.add_edge(n4)Add the directed
edges
n4.add_edge(n5)n1.add_edge(n2)
g.display()n2.add_edge(n3)
n3.add_edge(n1)
n3.add_edge(n4)
n4.add_edge(n5)

g.display()
```

Sample Output

```
Adjacency List:
1 -> 2
2 -> 3
3 -> 1, 4
4 -> 5
5 -> None
```

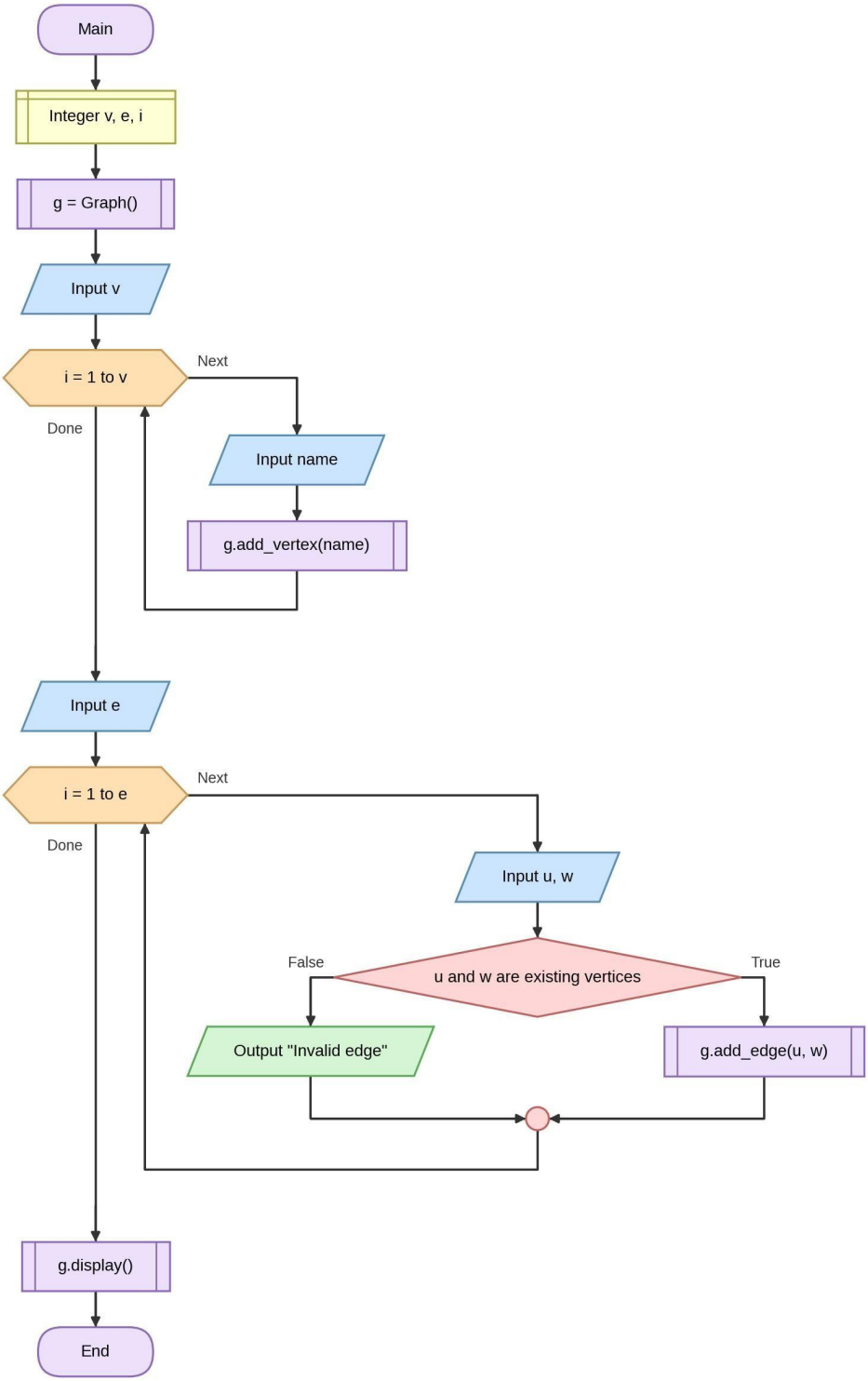
Back to Index

Program 20: Graph Creation Using User Input

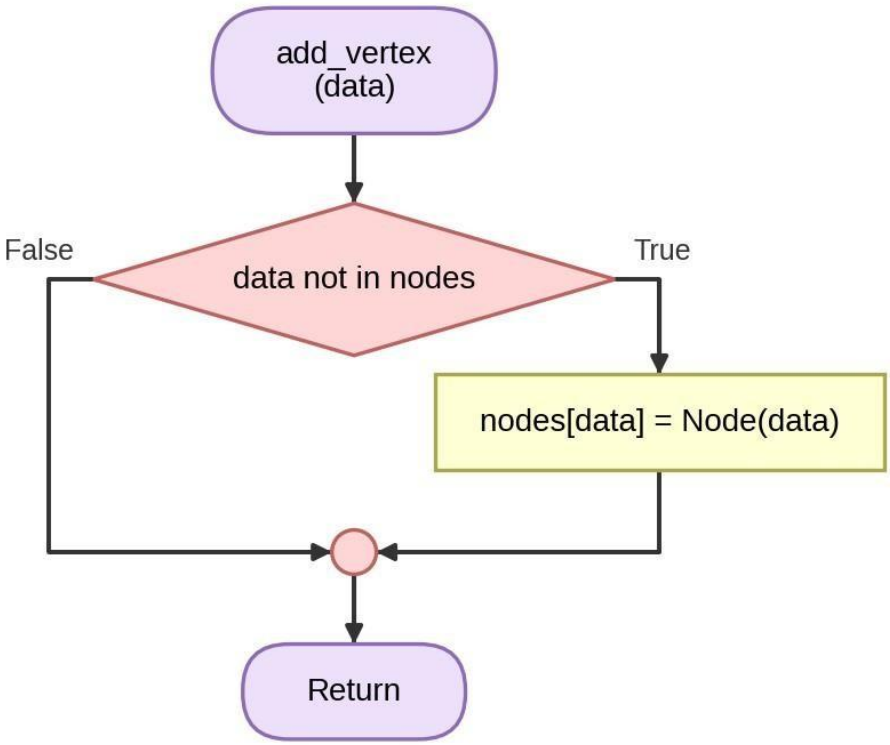
Aim

To write a Python program to create a graph by taking the number of vertices and edges from the user, reading the vertices and edges, and displaying the resulting graph using an adjacency list.

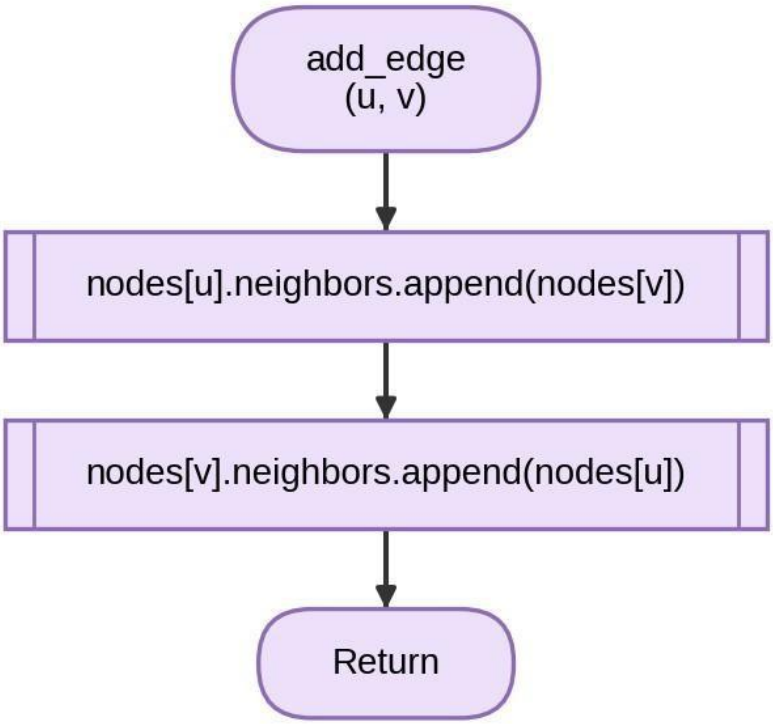
Flowchart



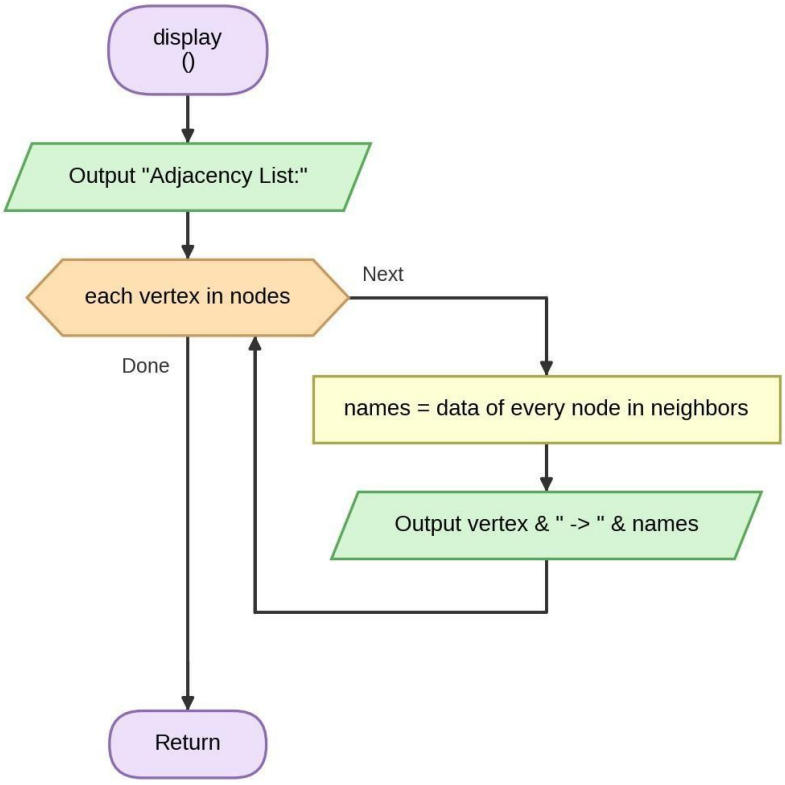
Main



Function add_vertex



Function add_edge (undirected)



Function display

Program

```
class Node:
    def __init__(self, data):
        self.data = data
        self.neighbors = []

class Graph:
    def __init__(self):
        self.nodes = {}

    def add_vertex(self, data):
        if data not in self.nodes:
            self.nodes[data] = Node(data)

    def add_edge(self, u, v):
        self.nodes[u].neighbors.append(self.nodes[v])
        self.nodes[v].neighbors.append(self.nodes[u])

    def display(self):
        print("Adjacency List:")
        for data, node in self.nodes.items():
            names = ", ".join(str(n.data) for n in node.neighbors) or "None"
            print(data, "->", names)

g = Graph()
v = int(input("Enter number of vertices: "))
```

```
        print("Adjacency List:")    for for i in
range(v):data, node in self.nodes.items():
    name = input("Enter vertex " + str(i + 1) + ": ")names = ", ".join(str(n.data) for n in node.neighbors) or
    g.add_vertex(name)
"None"                print(data, "->", names)
e = int(input("Enter number of edges: "))


gfor i in range(e):      u, w = input(=
Graph()
       "Enter edge " + str(i + 1)
v      + " (two vertices separated by space): "= int(input("Enter
number of vertices: "))    ).split()for i in range(v):
    if u in g.nodes and w in g.nodes:name = input("Enter vertex " + str(i + 1) + ": ")
       g.add_edge(u,
w)g.add_vertex(name)
    else:
print("Invalid edge!")
e      print("Both vertices must already exist.")=
int(input("Enter number of edges: ")) for i
in range(e):
g.display()
     u, w = input("Enter edge " + str(i + 1) + " (two vertices separated by space): ").split()
if u in g.nodes and w in g.nodes:
        g.add_edge(u, w)     else:         print("Invalid edge!
Both vertices must already exist.")

g.display()
```

Sample Output

```
Enter number of vertices: 4
Enter vertex 1: A
Enter vertex 2: B
Enter vertex 3: C
Enter vertex 4: D
Enter number of edges: 4
Enter edge 1 (two vertices separated by space): A B
Enter edge 2 (two vertices separated by space): A C
Enter edge 3 (two vertices separated by space): B D
Enter edge 4 (two vertices separated by space): C D
Adjacency List:
A -> B, C
B -> A, D
C -> A, D
D -> B, C
```

Back to Index